Swinburne University Of Technology

School of Science, Computing, and Engineering Technologies

ASSIGNMENT AND PROJECT COVER SHEET

Unit Code: COS 30049 Unit Title: Computing Technology Innovation Project

Assignment number and title: Assignment 3: Full-Stack Web Development for AI Application in Cybersecurity Scenarios Due date: 13 NOV 2026

Lab Group: Group 10 Tutor: Mohamad Faizal Alias Lecturer: Mohamad Faizal Alias

Family name: _____ Identity no: _____

Other names: _____

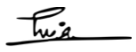
To be completed if this is an INDIVIDUAL ASSIGNMENT

I declare that this assignment is my individual work. I have not worked collaboratively nor have I copied from any other student's work or from any other source except where due acknowledgment is made explicitly in the text, nor has any part been written for me by another person.

Signature: _____

To be completed if this is a GROUP ASSIGNMENT

We declare that this is a group assignment and that no part of this submission has been copied from any other student's work or from any other source except where due acknowledgment is made explicitly in the text, nor has any part been written for us by another person.

ID Number	Name	Signature
105735572	Lalmuanpuia	
105960484	Livinggkumar A/L Supramaniam	
105965557	Teh Zhi Xiang	
105948781	Jean-Luc William Clovis	

Marker's comments:

Total Mark: _____

Extension certification:

This assignment has been given an extension and is now due on _____

Signature of Convener: _____ Date: /2025

1. Introduction

1.1 Purpose and Users

The purpose of this website is to provide an interactive road safety analysis platform that helps users understand road accident patterns and explore potential accident risks across different Local Government Areas (LGAs).

The system is designed for users such as researchers, students, road safety analysts, and members of the public who are interested in understanding road safety information. It provides data visualisations and machine learning features to support users in exploring accident trends, identifying similarities between areas and examining potential road accident risks.

1.2 What the Site Offers

The website provides four main sections to help users explore road safety information:

- **Home:** Provides an introduction to the project and an overview of the road safety information available in the system.
- **Risk Predictor:** Allows users to enter selected road safety characteristics to predict the potential accident risk using a Logistic Regression model.
- **Cluster Explorer:** Allows users to explore and compare LGAs based on their road safety characteristics using K-Means and DBSCAN clustering techniques.
- **About:** Provides information about the project, dataset, machine learning methods, model evaluation and system limitations.

These features combine machine learning and interactive visualisation to make road safety information easier to understand and explore.

1.1 Decisions Made

Several design and development decisions were made to ensure that the website is accessible, organised and easy to use.

React was selected as the frontend framework to support the development of reusable components and interactive user interfaces. A consistent navigation structure was implemented to allow users to move between the four main sections of the website easily.

The website also uses a simple and organised layout to present road safety information clearly. Different sections were separated according to their functionality so users can access the required information without unnecessary navigation.

Interactive charts and visualisations were included to help users understand road accident patterns more effectively instead of relying only on numerical data and text.

1.4 Limitations

Although the website provides useful tools for exploring road safety information, it has several limitations. The accuracy of the risk predictions depends on the quality and availability of the dataset and the features used to train the machine learning model. Road infrastructure, traffic conditions, and driver behavior are just some of the factors that the system might not take into consideration.

Furthermore, the risk prediction model uses predefined risk categories, which may not represent every possible level of accident risk. The clustering results also depend on the selected features and algorithm configurations, meaning that different settings may produce different groupings.

The website is designed to support analysis of road safety and provide information about accident statistics. However, its predictions should not be considered to be guarantee results or utilized in place of professional evaluations of road safety.

1.5 Frontend Development and User Experience

The frontend of the Road Safety Analytics website was developed using React.js, with Vite as the development environment, CSS for styling, and Axios for communication with the FastAPI backend. The website consists of four main sections: Home, Risk Predictor, Cluster Explorer, and About, providing users with clear navigation and easy access to the system's functionalities. The Risk Predictor page allows users to enter relevant road safety information and obtain accident risk predictions through the integrated machine learning model. User experience was considered by implementing a consistent design, organised layouts, clear navigation, and appropriate feedback messages. Responsive CSS styling was also applied to ensure that the website can be accessed across desktop, tablet, and mobile devices. Overall, the frontend provides an accessible and user-friendly interface for presenting road safety insights and interacting with the prediction system.

2. System Architecture

2.1 Architecture Diagram

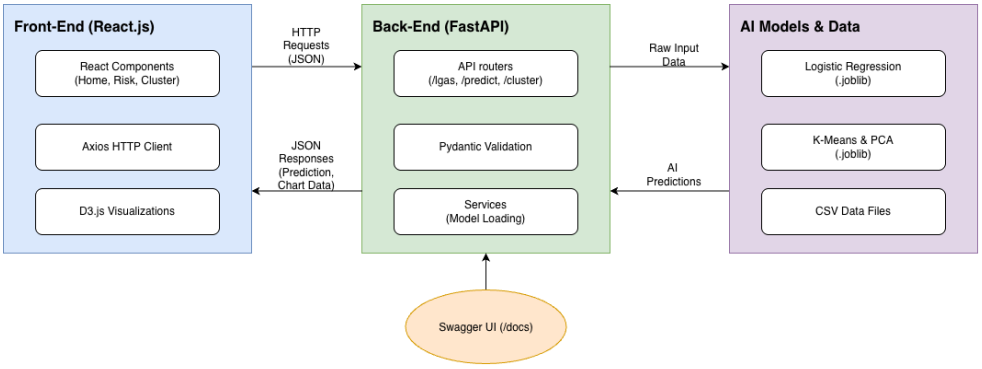


Figure 1: The Architecture Diagram

The diagram illustrates how the different parts of our system work together. We made sure to keep the frontend, backend, and AI models separate from each other. The part that users interact with, built using React, only focuses on what's visible to them. It doesn't have any knowledge of the inner workings of Scikit-Learn or Pandas. All it does is send information to the API and then wait for a response in JSON format. This separation was crucial because it allowed us to work on the frontend and backend simultaneously without interfering with each other's work. By doing so, we could make changes and updates without breaking anything.

2.2 Technology Choices

1. **React**: React is used to build a single-page application. It is in charge of managing the state, so when a user changes a filter or moves a slider, React updates the data and then tells D3 to redraw the chart. This all happens without the page reloading, which is pretty handy.

2. **FastAPI:** We use FastAPI for the backend. It handles the HTTP requests that come in, checks the data to make sure it is correct, and then runs our AI models. After that, it sends the results back to React in a JSON format.

3. **D3.js:** We used D3.js to make all the charts. Since we created them from scratch using SVG elements, we got to decide exactly how they looked and worked. For example, we could add special tooltips that pop up when a user hovers over any of the hundreds of dots.

4. **Scikit-Learn:** Scikit-learn is a library for machine learning that we used for the previous assignment to train our models. For this project, we didn't rebuild any models from scratch. Instead, we used the .joblib files we saved earlier from Assignment 2 and loaded them into the system.

5. **Axios:** We use Axios in React to send HTTP requests to the FastAPI backend.

2.3 Data Flow for One Prediction

Here is the expected data flow when the React frontend sends a request to the Risk Predictor API:

1. **Client validation (Frontend):** The React form validates that all fields are present before sending the request.

2. **The POST request:** React uses Axios to send a JSON object to POST /predict (e.g., {"population": 50000, "severe_pct": 60, "wet_weather_pct": 15, "avg_speed_limit": 80}).
3. **Server validation:** FastAPI immediately checks the incoming JSON against the RiskRequest Pydantic schema. If a field is missing or a percentage is outside the 0 to 100 range, the backend rejects the request and returns a 422 Unprocessable Content error before calling the AI model.
4. **Preprocessing:** The backend passes the raw numbers into the Scikit-Learn pipeline. The pipeline automatically applies the StandardScaler to normalise the inputs.
5. **The model:** The Logistic Regression model calculates the probability of the LGA being high risk.
6. **Postprocessing:** The backend checks the probability against the decision threshold (0.5). It labels the result as "High" or "Low" risk and rounds the probability to 4 decimal places.
7. **The JSON response:** The backend sends the final JSON back to React, containing the risk level, the probability, and the original inputs that were received.

2.4 Project Structure

The project is split into two main folders: **backend** and **frontend.**

- **backend/app/routers/:** The FastAPI endpoint definitions (lgas.py, risk.py, cluster.py).
- **backend/app/schemas/:** The Pydantic models used for validating input (e.g., RiskRequest).
- **backend/app/services/:** The logic that loads the .joblib models and runs the predictions.
- **backend/models/:** The saved .joblib files for the Risk, K-Means, and PCA models.
- **backend/data/:** Contains all the CSV files. This includes the cleaned historical LGA data and the exported model evaluation metrics that feed the D3 charts.
- **frontend/src/**: Contains all the React code (pages, components, D3 charts, and CSS).

2.5 Communication & CORS

During development, we ran the frontend and backend on two different ports. React was on localhost:5173, and FastAPI was on 127.0.0.1:8000.

Because they are on different ports, the browser treats them as two different websites and blocks requests between them by default. To get around this, we set up CORS (Cross-Origin Resource Sharing) in the FastAPI backend. We added http://localhost:5173 to the list of allowed origins, and set allow_methods and allow_headers to "*" so that Axios could send POST requests with JSON bodies.

Without this setup, every time the React app tried to fetch data from the API, the browser would have blocked it and thrown a CORS error in the console. We confirmed this

by testing the frontend on an unauthorized port (localhost:3000), where the browser blocked the request and the frontend displayed an error message instead of crashing.

2.6 Limitations

The backend loads all models and CSV data into memory at startup. This makes the API very fast, but it means any changes to the data require restarting the server. There is also no database or authentication layer, so the application is read-only and does not save any user data.

3. Data and Files

This section outlines the raw data sources used to train the models and lists the specific files that were generated and used by the web application to populate the charts and make predictions.

3.1 Sources

The project relies on two primary raw datasets from Assignment 2:

- **NSW Crash Data:** The road crash records were obtained from Transport for NSW, covering the period from 2020 to 2024. This dataset provided the location, severity, and conditions of individual crashes (Transport for NSW, n.d.).
- **ABS Population Data:** To calculate crash rates per 100,000 residents, we used the Estimated Resident Population data from the Australian Bureau of Statistics (Australian Bureau of Statistics, 2026).

3.2 Processing

The data processing was conducted in Assignment 2 and exported as CSV files for this web application. We removed records with "Unknown" values and filtered out any data outside the 2020–2024 window. The crash data was then aggregated by Local Government Area (LGA) and Year, and merged with the ABS population figures. This resulted in a final, clean dataset of 129 NSW LGAs across 5 years, producing exactly 645 LGA-year rows that the application uses to train the risk model and draw the charts.

3.3 Files Used by the Application

File Name	Type	Used By
lga_yearly_aggregates_clustering.csv	Data	GET /lgas, POST /api/classification/predict, POST /cluster
lga_cluster_points.csv	Data	GET /api/clusters/pca
cluster_profiles.csv	Data	GET /api/clusters/profiles

logreg_metrics.csv	Data	GET /api/classification/metrics
logreg_test_predictions.csv	Data	GET /api/classification/predictions
logreg_permutation_importance.csv	Data	GET /api/classification/importance
logreg_coefficients.csv	Data	GET /api/classification/coefficients
lga_risk_model.joblib	Model	POST /api/classification/predict
kmeans_lga.joblib	Model	POST /cluster

3.4 Limitations

Crash rates per resident are statistically unstable for very small LGAs. In a town with a population of 2,000, a single extra crash can significantly alter the crash rate per 100,000, which can lead to volatile predictions from the AI model.

4. Home Page & App Shell

This section describes the development of the frontend structure of the Road Safety application. It focuses on the application layout, page routing, navigation and responsive design to provide users with a consistent and user-friendly experience.

4.1 Application Shell & Routing

The application shell was developed using React to provide a consistent structure across the Road Safety application. It acts as the main framework of the website, allowing users to access different sections through a shared layout.

The application consists of four main pages which are Home, Risk Predictor, Cluster Explorer and About. React Router was implemented to manage navigation between these pages without requiring a full page reload. Each page is assigned its own route, allowing users to access the required functionality easily.

The shared application structure also helps maintain consistency throughout the website by using common layout elements such as the header, navigation menu and main content area. This makes the application easier to navigate and maintain.

4.2 Navigation & Responsive Layout

The navigation menu was designed to allow users to access the main sections of the Road Safety application easily. It provides links to the Home, Risk Predictor, Cluster Explorer and About pages.

The navigation layout is kept consistent across the application so users can move between different sections without confusion. Each navigation link directs users to its corresponding page, making the system easier to use.

Responsive design was also considered to ensure that the application can be accessed on different screen sizes, including desktops, tablets and mobile devices. CSS styling and media queries were used to adjust the layout and content according to the screen size. This helps maintain readability and usability across different devices.

4.3 Home Page Design

The Home page serves as the main landing page of the Road Safety Analytics website. It was designed to introduce the purpose of the platform and provide users with easy access to its main features. The page consists of two main sections: the Hero Section and the Overview Section.

The Hero Section displays the main heading, "Understanding Road Safety Through Data", together with a short description explaining the purpose of the website. It also includes a background gradient design and an aerial image of busy city highway traffic to visually represent the road safety theme. An "Explore Risk Predictor" button is provided to allow users to navigate directly to the Risk Predictor page.

The Overview Section introduces the available platform features through three interactive feature cards which is Risk Predictor, Cluster Explorer, and About. Each card contains an icon, a short description, and a navigation link. Users can select the Risk Predictor card to explore accident risk predictions, the Cluster Explorer card to examine groups of areas with similar road safety characteristics, or the About card to learn more about the dataset, analytical methods, machine learning models, and project limitations.

The overall design uses a dark colour scheme with a combination of navy, grey, and purple tones to create a consistent and modern appearance. CSS styling is applied to the navigation elements, buttons, feature cards, spacing, and hover effects to improve the visual presentation and user interaction. Responsive CSS media queries are also implemented to

adjust the layout for tablet and mobile screen sizes. This ensures that the Home page remains accessible and easy to use across different devices.

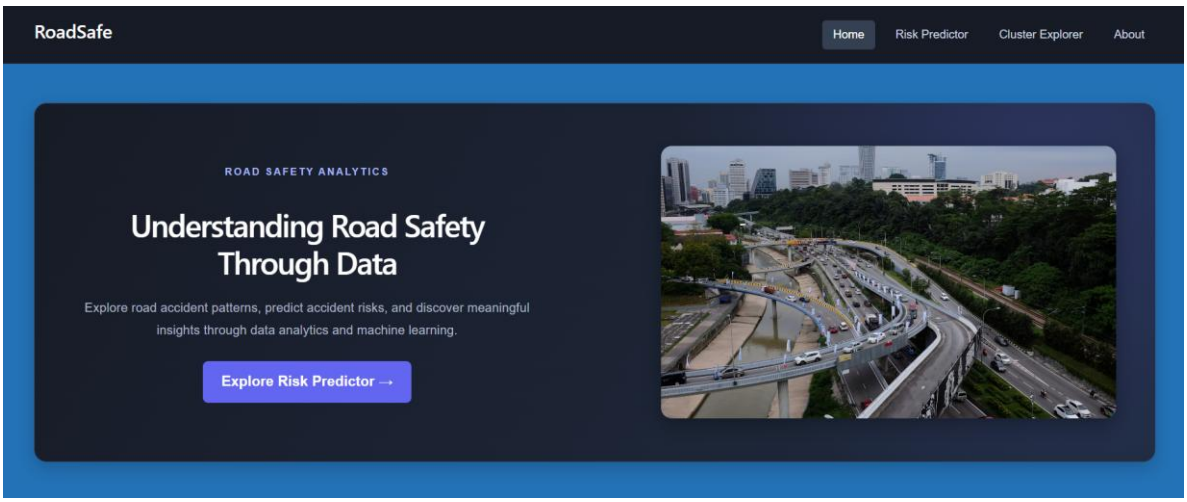


Figure 2:Home page

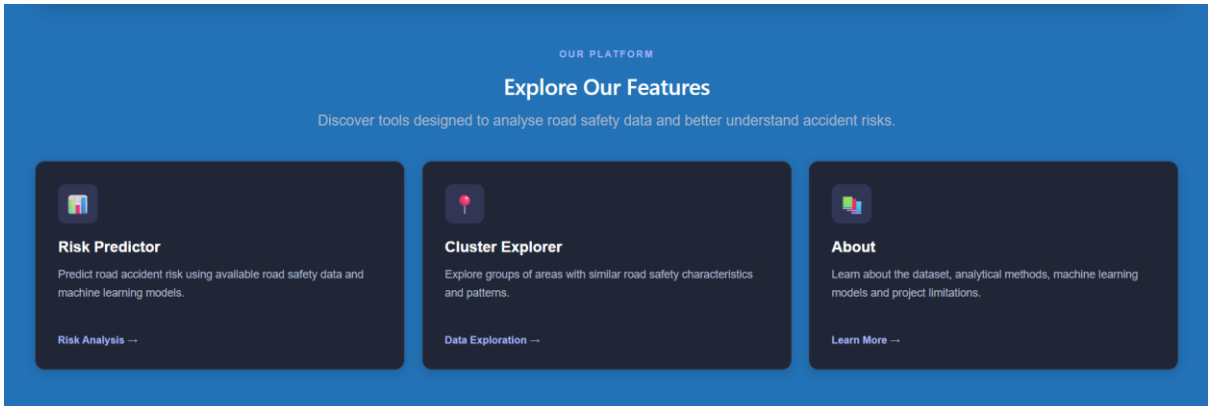


Figure 3:Home page cards

4.4 Responsive Design (Tablet and Mobile)

Tablet (768px)

The home page adjusts its layout to fit the tablet screen, ensuring that the content, navigation and visual elements remain properly arranged and readable.

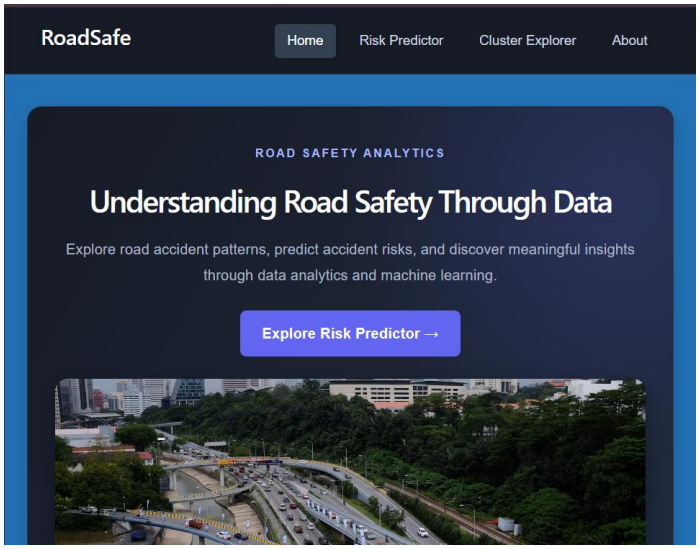


Figure 4 : Home page responsiveness 1

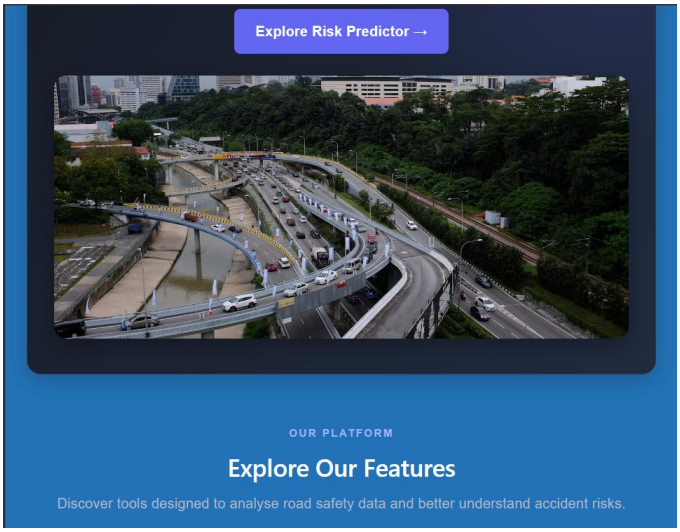


Figure 5 : Home page responsiveness 2

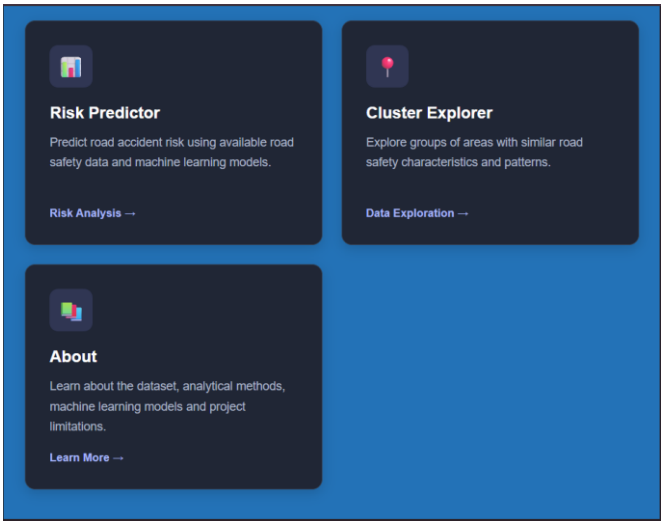


Figure 6 : Home page responsiveness 3

Mobile 375px

The home page adapts to a smaller mobile screen by adjusting the layout, text sizes and spacing to provide a clear and user-friendly viewing experience.

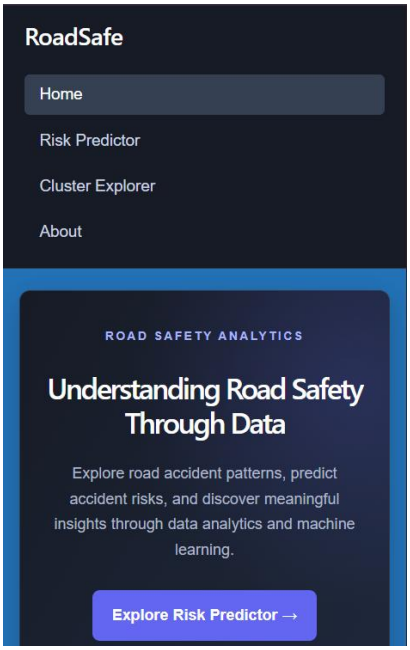


Figure 7: Home page responsiveness 4

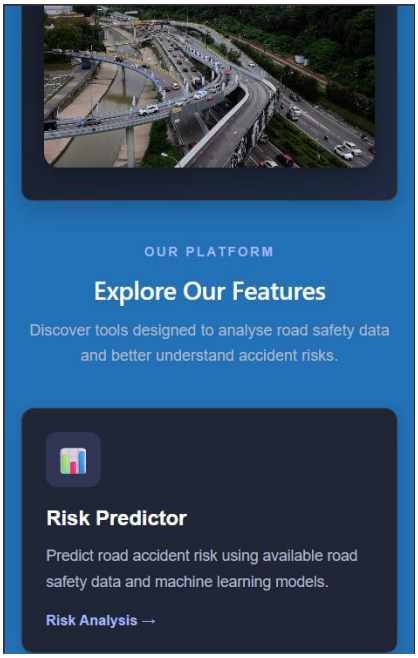


Figure 8: Home page resposnivesness 5

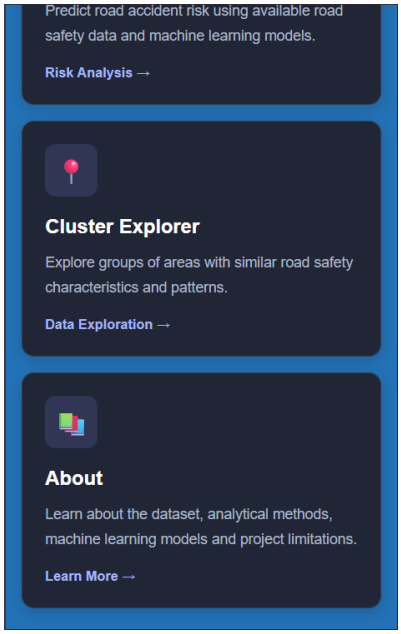


Figure 9 : Home page responsiveness 6

4.5 About Page Layout

The About page was designed to provide users with an overview of the Road Safety Analytics website, its objectives, dataset information, and machine learning methodologies. The page uses a consistent dark-themed layout to maintain visual consistency with the rest of the website. Information is organised into separate sections and cards to improve readability and make the content easier to navigate.

The page includes an introduction to the project, its main objectives, an overview of the dataset features, and explanations of the machine learning techniques used, including Logistic Regression, K-Means, DBSCAN, and Principal Component Analysis (PCA). It also

provides information about the clustering visualisations and the system's limitations. The responsive layout allows the content to adjust to different screen sizes, providing a more user-friendly experience across desktop and mobile devices.

4.6 About Page Contents (Model Metrics)

The About page also presents the evaluation results of the Logistic Regression risk prediction model to help users understand its performance. The model was evaluated using several classification metrics, including accuracy, precision, recall, and F1-score.

The model achieved an accuracy of 80.7%, indicating that approximately 80.7% of the test predictions were correct. For Class 1, the model recorded a precision of **70.6%**, a recall of **77.4%**, and an F1-score of **73.8%**. These metrics provide a better understanding of how effectively the model identifies high-risk cases.

A confusion matrix is also included to show the comparison between actual and predicted classifications. Out of 129 test observations, the model correctly classified 47 Class 0 cases and 48 Class 1 cases. It incorrectly classified 20 Class 0 cases as Class 1 and 14 Class 1 cases as Class 0.

Presenting these evaluation results allows users to assess the model's predictive performance and understand its strengths and limitations. Overall, this section improves transparency by providing measurable evidence of how well the risk prediction model performs.

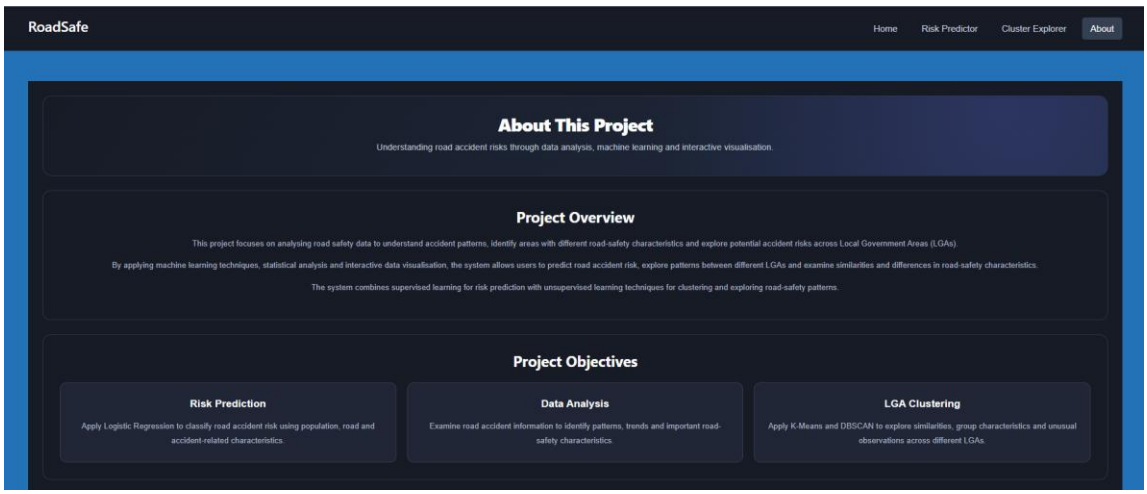


Figure 10 : About page header



Figure 11: Home page dataset info

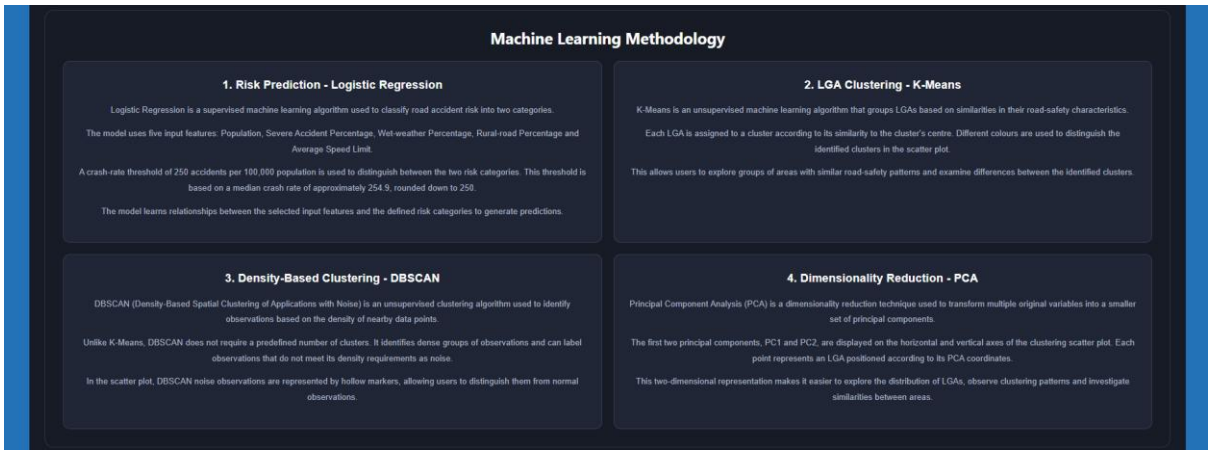


Figure 12: About page ML



Figure 13: About page Cluster View

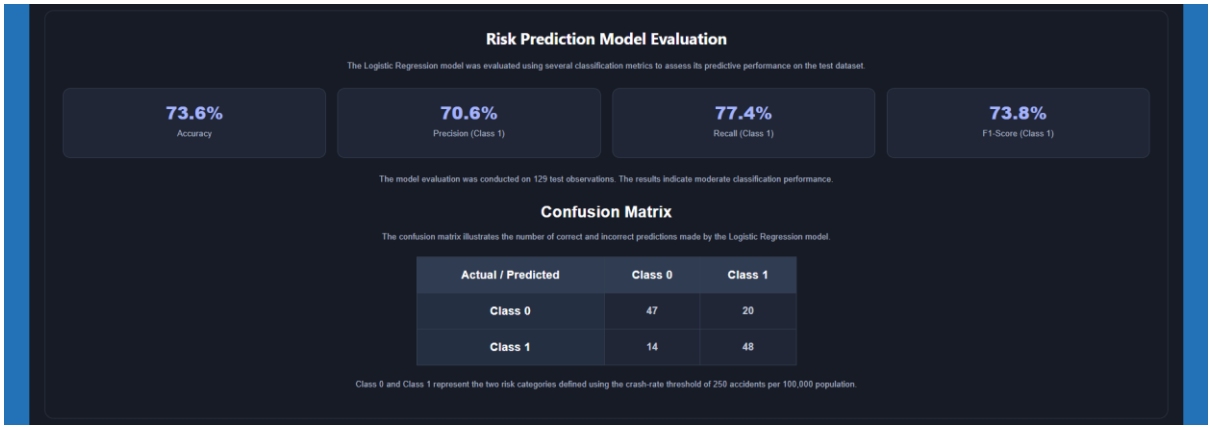


Figure 14 : About page Risk Model

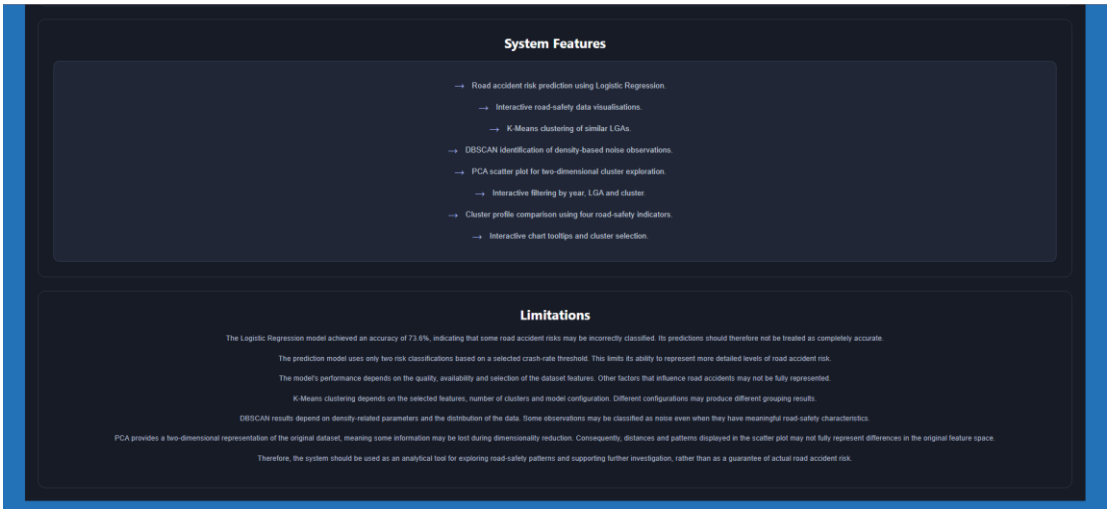


Figure 14: About page Footer

4.7 Responsive Design (Tablet and Mobile)

Tablet (768px)

The About page uses a responsive layout to fit the tablet screen, adjusting the content sections, information cards and grids for better readability.

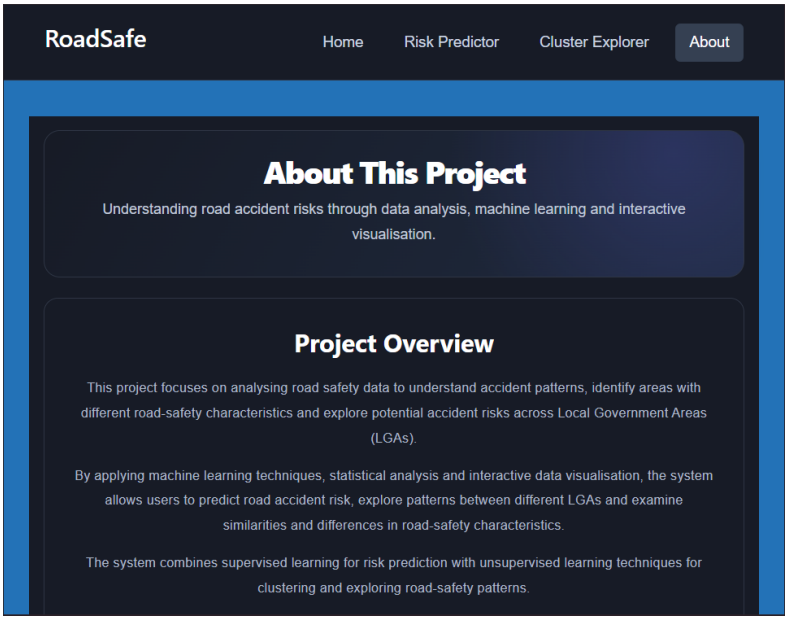


Figure 15 : About page responsive 1

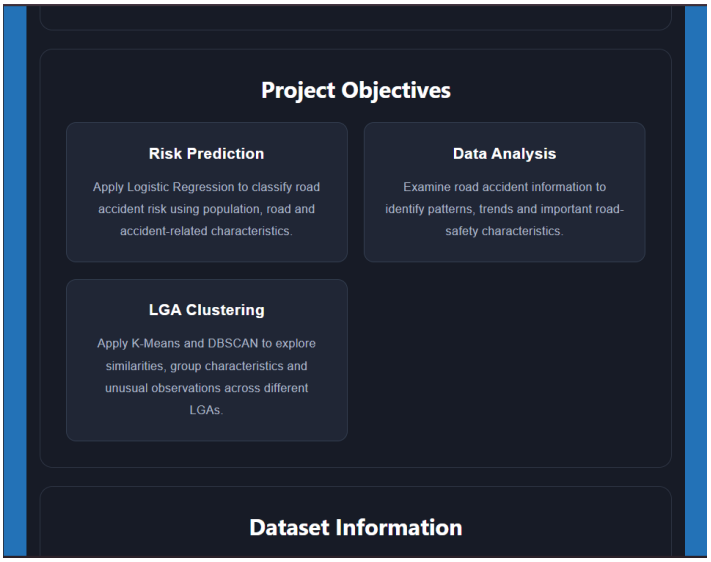


Figure 16: About page responsive 2

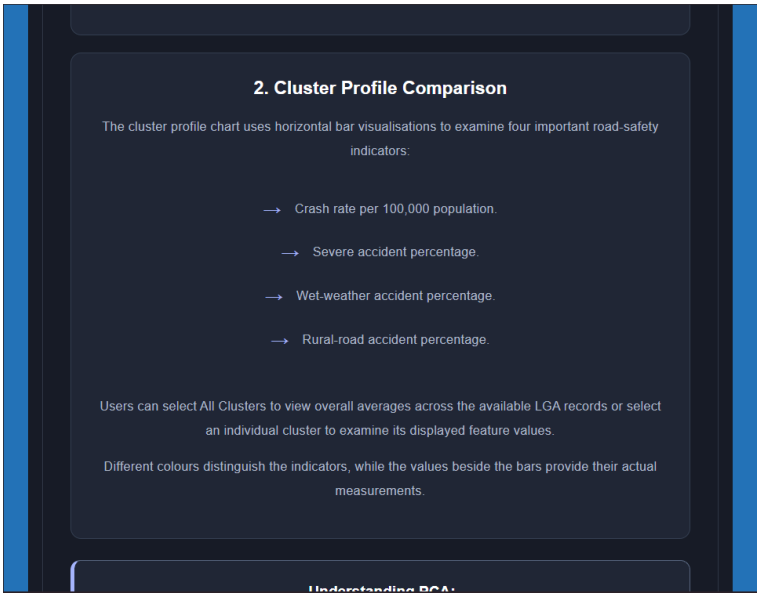


Figure 17 : About page responsive 3

Mobile (375px)

The About page rearranges its content into a single-column layout, making the information, cards and sections easier to view and navigate on mobile screens.

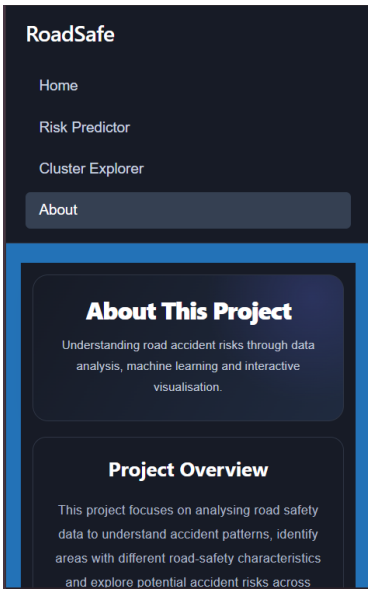


Figure 18 : About page responsive 4

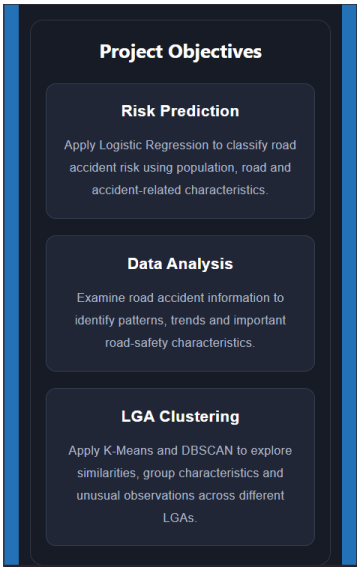


Figure 19 : About page responsive 5

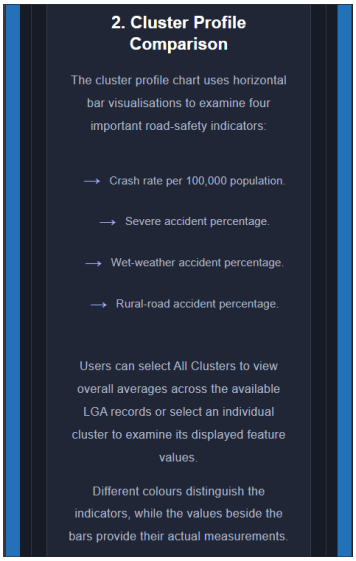


Figure 20 : About page responsive 4

5. Risk Predictor Page

This section describes the development of the Risk Predictor page, which allows users to estimate the potential road accident risk of a Local Government Area (LGA) based on selected road safety characteristics. The page combines a user input form, client-side validation, Axios integration with the FastAPI backend, and interactive D3.js charts to present the Logistic Regression model results in a clear and accessible way.

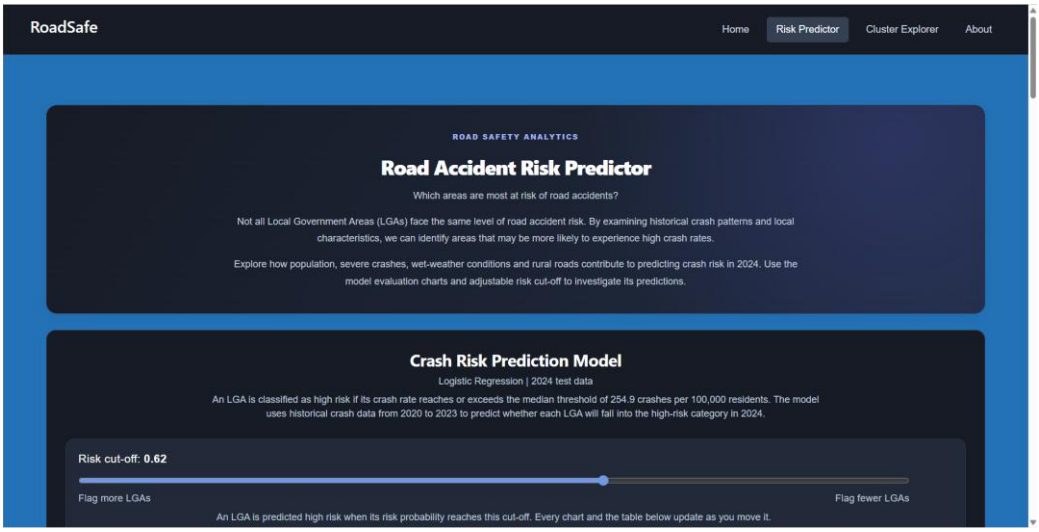


Figure 21 : Road Risk Page Header

5.1 Page Layout & Input Form

The Risk Predictor page was designed to provide users with a simple and guided interface for exploring accident risk predictions. The page is organised into two main areas: a showcase of the model's testing results and an interactive form that demonstrates the model in action.

After a short introductory section, users can explore the results of the crash risk prediction model and see how accurately it performs. We also provided a ROC curve and a confusion matrix to help visualise the model's performance. In addition, a feature importance chart shows which features have the greatest impact on the predictions. Users can also adjust the probability threshold themselves to see how it affects the model's accuracy and classification results.

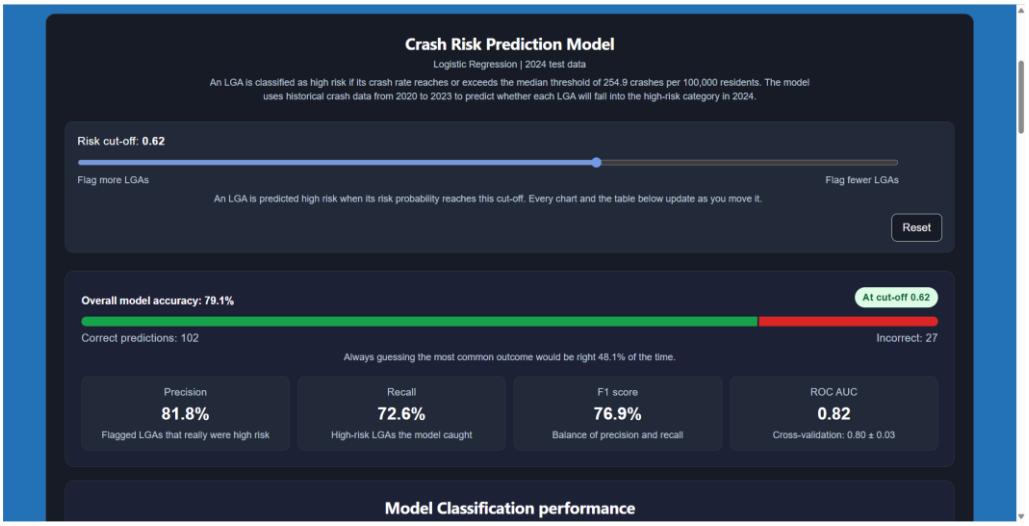


Figure 22 : Road Risk Prediction Model

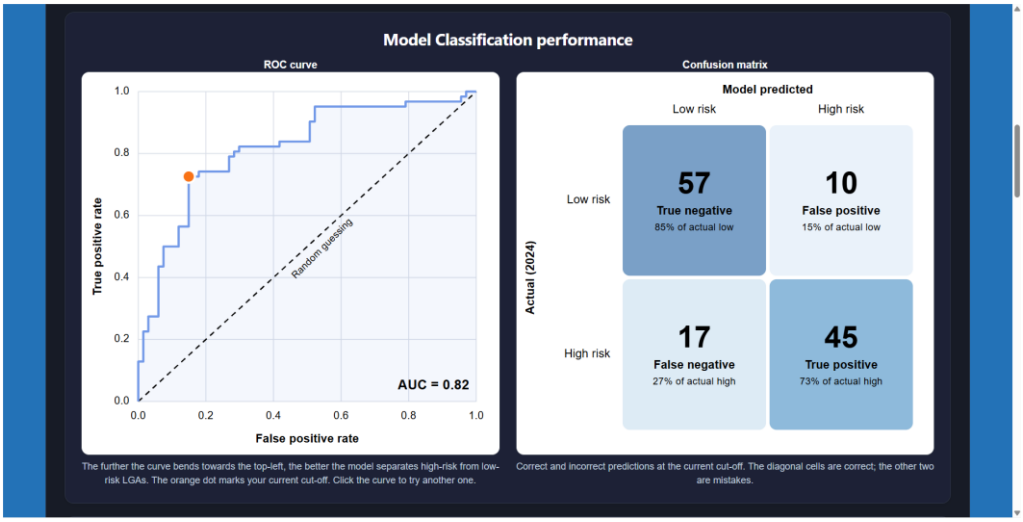


Figure 23 : Model Classification

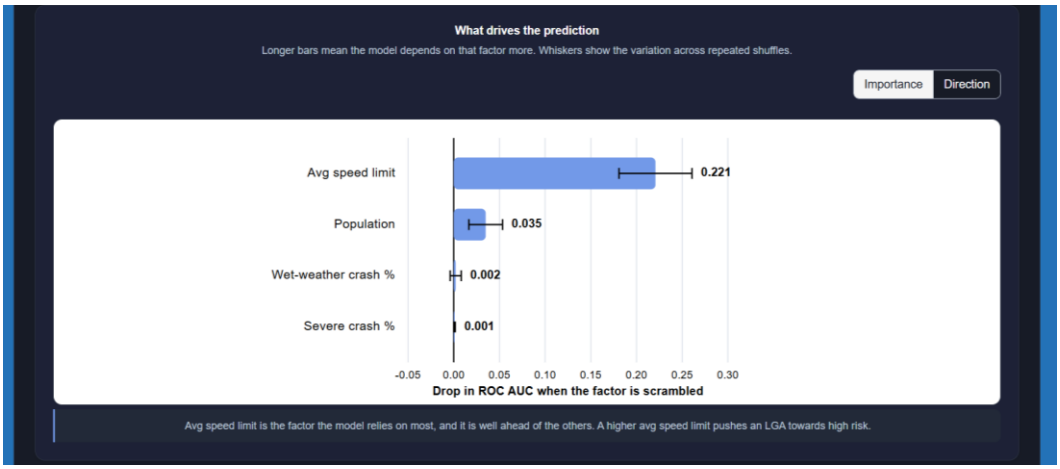


Figure 24: Prediction Drive data

In addition, we implemented a simple input form to demonstrate the Logistic Regression model in action. The input form contains four numerical fields that represent the key road safety features used to train the model:

- **Population**: The estimated resident population of the LGA.
- **Severe %:** The percentage of crashes that resulted in a injury or fatality.
- **Wet-weather %**: The percentage of crashes that occurred during wet weather conditions.
- **Avg speed limit:** The average posted speed limit of the road where the crash occured

Each field is implemented as a controlled React input, meaning that the form state is managed using the useState hook. This allows the application to track changes as users enter their values and update the form accordingly. After filling in the four inputs, users can also provide an optional LGA name and adjust the probability cut-off. They can then press the "Predict Crash Risk" button at the bottom of the form, which sends a prediction request to the backend API.

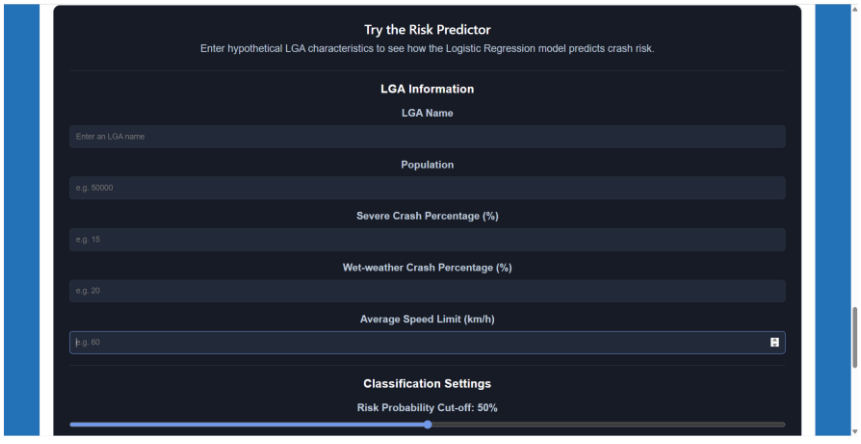


Figure 25: Risk Prediction form

The results panel displays the predicted risk level, probability score and probability threshold used by the model, alongside a short explanation to help users understand the prediction.

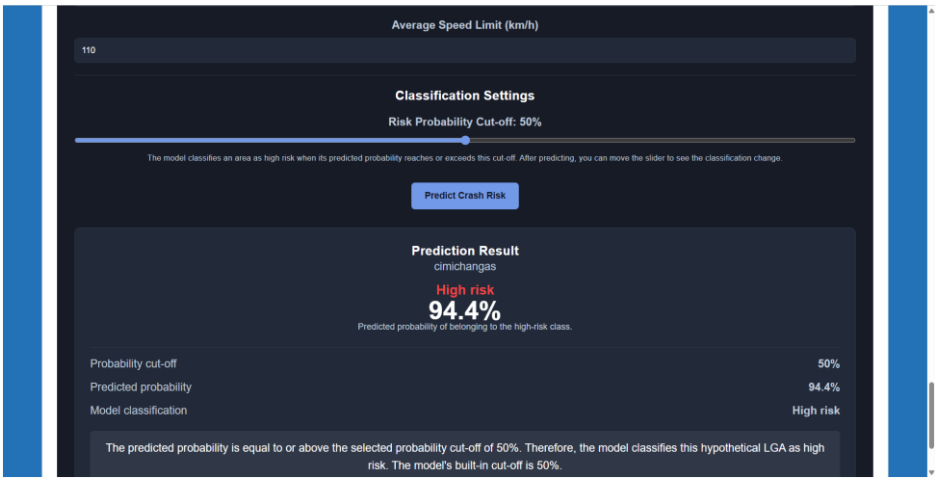


Figure 26 : Risk Prediction form results

5.2 Client-side Validation

Client-side validation was implemented to ensure that users provide valid inputs before a request is sent to the backend. This improves the user experience by catching errors early and reducing unnecessary API calls.

Validation rules were applied to each input field:

- Population must be a positive number greater than zero.
- Severe Crash Percentage must be a number between 0 and 100.
- Wet-Weather Crash Percentage must be a number between 0 and 100.
- Average Speed Limit must be a positive number.

If a user enters a value that violates any of these rules, an error message is displayed directly below the corresponding field. The "Predict Crash Risk" button is also disabled until the error is corrected. The input fields use the HTML type="number" attribute, which restricts non-numeric input in most browsers.

This client-side validation works alongside the server-side validation implemented in FastAPI (described in Section 7.3). While the client-side checks prevent most invalid requests from being sent, the backend still validates all incoming data using Pydantic constraints to ensure that the model never receives out-of-range values.

5.3 Fetch API Integration

Fetch API was used to handle communication between the React frontend and the FastAPI backend. When the user clicks the "Predict Crash Risk" button, the application sends

a POST request to the /api/classification/predict endpoint with a JSON body containing the four input values.

The request is structured as follows:

const response = await fetch(`${API_URL}/api/classification/predict`, {

method: "POST",

headers: { "Content-Type": "application/json" },

body: JSON.stringify({

population: Number(formData.population),

severe_pct: Number(formData.severe_percent),

wet_weather_pct: Number(formData.wet_weather_percent),

avg_speed_limit: Number(formData.avg_speed_limit)

})

});

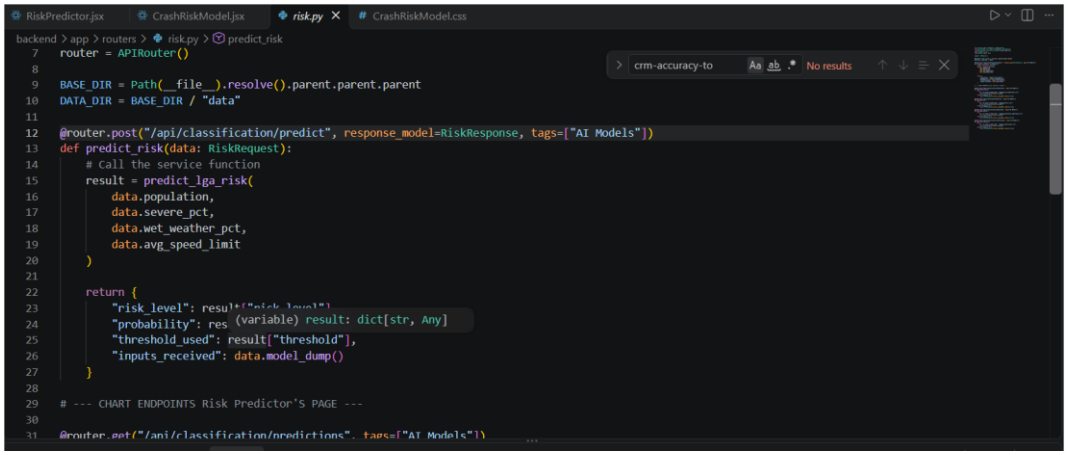


Figure 27 : API integration

The API base URL is constructed dynamically using window.location.hostname with port 8000, allowing the frontend to connect to the backend regardless of the host environment. This approach was chosen to support both local development and deployment without requiring code changes.

The response from the backend is a JSON object containing the risk_level, probability and threshold_used values. These values are stored in React state and displayed in the results panel. A loading state was also implemented so that the button displays a "Predicting..." message while the request is in progress, preventing users from submitting multiple requests at once.

Error handling was also included to ensure that any network or server errors are displayed to the user in a clear and friendly way. If the request fails, an error message is shown in the results panel instead of allowing the application to crash.

5.4 The Logistic Regression Model

The Risk Predictor uses a Logistic Regression model that was trained in Assignment 2 and saved as a .joblib file (lga_risk_model.joblib). Logistic Regression was chosen because it is well-suited to binary classification problems, where the goal is to predict whether an LGA is classified as "High" risk or "Low" risk based on its road safety characteristics.

The cleaned LGA-year dataset contains 645 records across 129 NSW LGAs and 5 years (2020-2024). The model was trained on the 2020-2023 records (516 records) and tested on the 2024 records (129 records). The target variable was created by labelling each LGA-year record as either high risk or low risk, based on whether its crash rate was 250 or more crashes per 100,000 population. This value is approximately the median crash rate (254.9), which keeps the two classes balanced. The four input features used by the model are:

- Population
- Severe crash percentage
- Wet-weather crash percentage
- Average speed limit

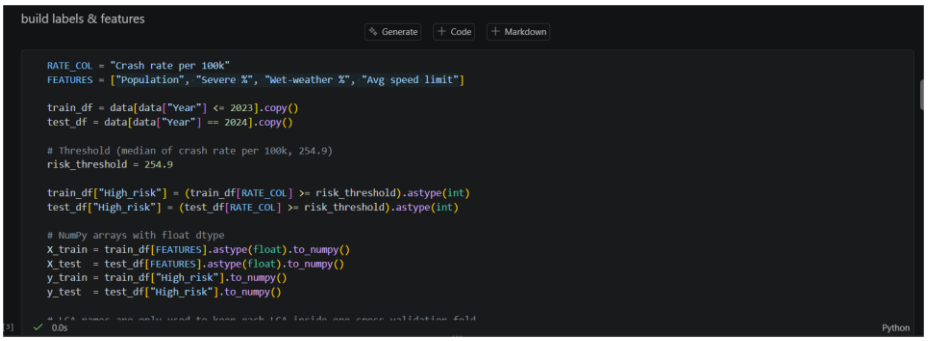


Figure 28 : Trained model

Before training, the features were normalised using a StandardScaler, which transforms each feature so that it has a mean of 0 and a standard deviation of 1. This step is important because the features have very different scales (e.g. population is in the tens of thousands, while percentages are between 0 and 100). Without scaling, features with larger numerical values could have a greater influence on the model.

The scaler and the model were combined into a single Scikit-Learn Pipeline, which was then saved as a .joblib file. When a prediction request is received, the backend loads the pipeline and passes the raw input values through it. The pipeline automatically applies the StandardScaler before calling predict_proba() on the Logistic Regression model to calculate the probability that the LGA is high risk.

The model's performance was evaluated using accuracy, precision, recall, F1-score

and a confusion matrix, which are presented on the About page (see Section 4.5). The model

initially achieved an accuracy of 76.7%, which increased to 80.7% after adjusting the probability threshold. This indicates that the model correctly classified approximately four out of five test cases at the adjusted threshold.

5.5 D3.js Charts

To help users understand the model's performance and behaviour, three D3.js charts were implemented on the Risk Predictor page. These charts are rendered as SVG elements when the page loads. The ROC curve and confusion matrix also update whenever the user adjusts the probability threshold.

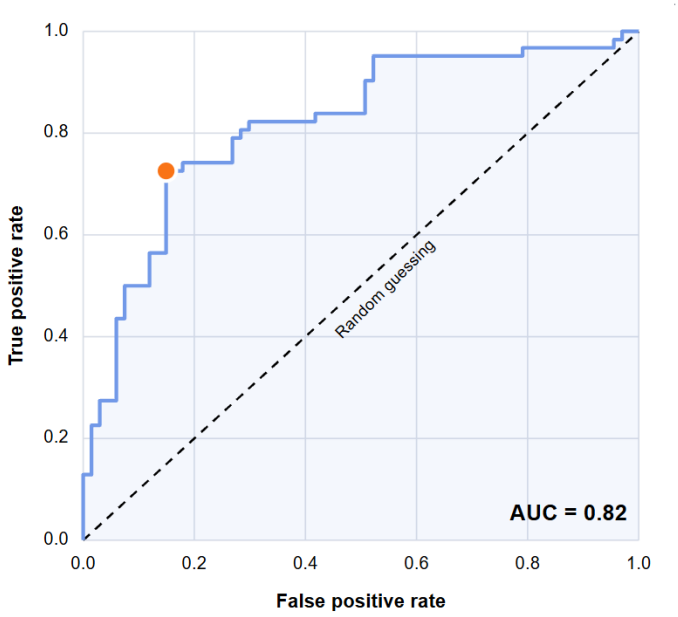


Figure 29 : D3 False Postive rates

ROC AUC Curve: The Receiver Operating Characteristic (ROC) curve shows the trade-off between the true positive rate and the false positive rate at different classification thresholds. The chart is built using the logreg_test_predictions.csv file, which contains the predicted probabilities and true labels for the test set. An orange marker indicates where the currently selected probability threshold sits on the curve. The area under the curve (AUC) is displayed as a summary metric, allowing users to assess how well the model distinguishes between high-risk and low-risk LGAs. The model achieved an AUC of 0.82, compared with 0.5 for random guessing, represented by the dashed diagonal line.

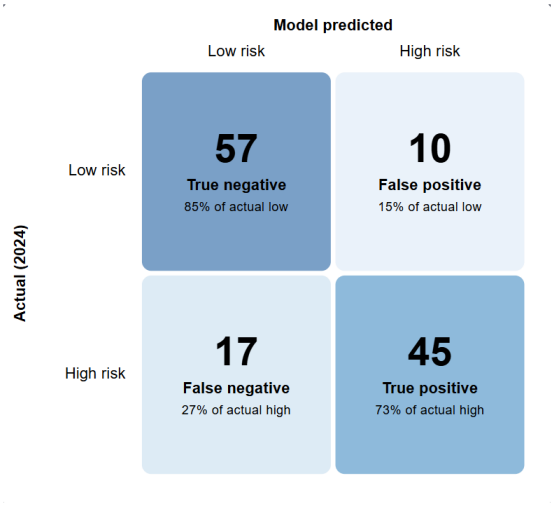


Figure 30 : D3 Model Predicted

Confusion Matrix: This chart compares the model's predictions for the 129 LGAs in the 2024 test set with their actual risk levels. Each cell displays a count representing one of four outcomes: true negatives (low-risk LGAs correctly predicted as low risk), false positives (low-risk LGAs incorrectly predicted as high risk), false negatives (high-risk LGAs incorrectly predicted as low risk) and true positives (high-risk LGAs correctly identified).

In the example shown, the model correctly identified 57 low-risk and 45 high-risk LGAs, while 10 low-risk LGAs were incorrectly flagged as high risk and 17 high-risk LGAs were missed. The confusion matrix updates whenever the probability threshold is adjusted, allowing users to observe the trade-off between identifying more high-risk LGAs and producing more false alarms.

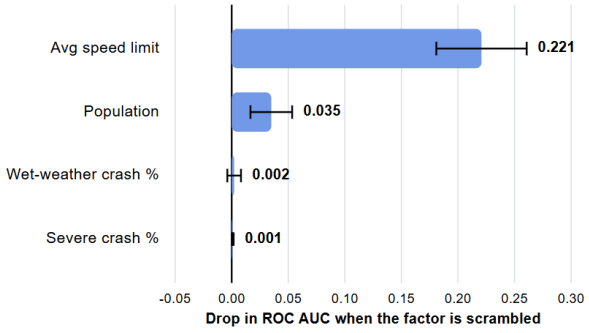


Figure 31 : D3 Confusion Matrix

Feature Importance Chart: This chart displays the permutation importance of each feature, showing how much the model's ROC AUC decreases when a feature's values are randomly shuffled. The data is loaded from logreg_permutation_importance.csv, and the whiskers show the variation across repeated shuffles. Features with higher importance have a greater influence on the model's predictions, helping users understand which road safety characteristics matter most. In the chart, average speed limit has by far the largest influence (0.221), followed by population (0.035), while the wet-weather and severe crash percentages have almost no effect (0.002 and 0.001).

D3.js was chosen because it provides fine-grained control over SVG elements, allowing the charts to be styled precisely and to include interactive tooltips that display additional information when the user hovers over a data point. The charts are also responsive, adjusting their dimensions based on the container width to ensure they remain readable on smaller screens.

5.6 Responsive Design

The risk predictor uses CSS responsiveness to adjust the page layout and chart components according to different mobile devices. So, two breakpoints were implemented at 900px and 550px to provide a better layout for users on tablets and phones.

Tablet responsiveness (900px)

For screens with a width of 900px and below, CSS padding is reduced to make better use of the available screen space. This allows the chart containers and page sections to fit within the narrower display. The ROC Curve chart and Confusion Matrix are also reduced in size to accommodate the smaller screen while keeping the charts readable. The LGA table is slightly compressed to fit the tablet screen, allowing users to view most of its contents without excessive horizontal scrolling.

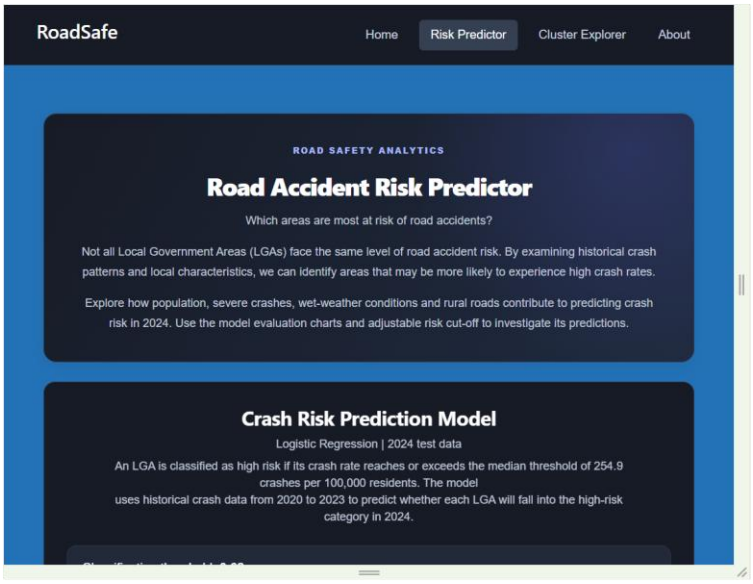


Figure 32 : Risk Predictor Responsiveness 1

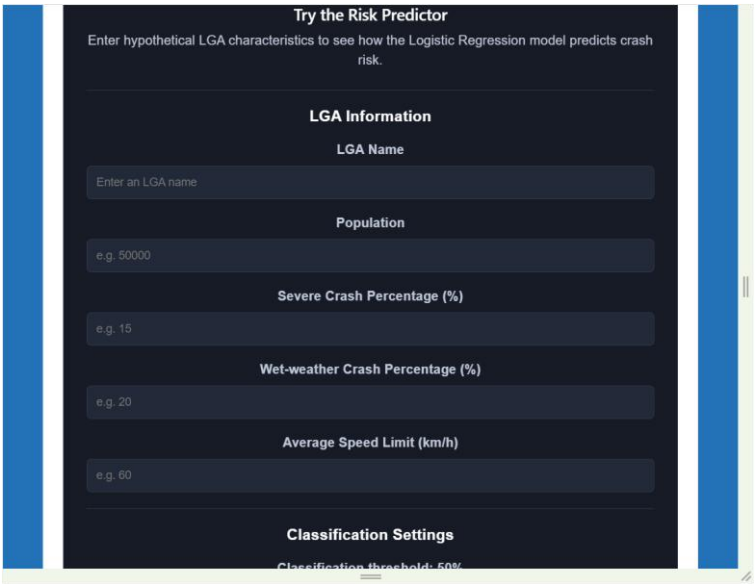


Figure 33: Risk Predictor Responsiveness 2

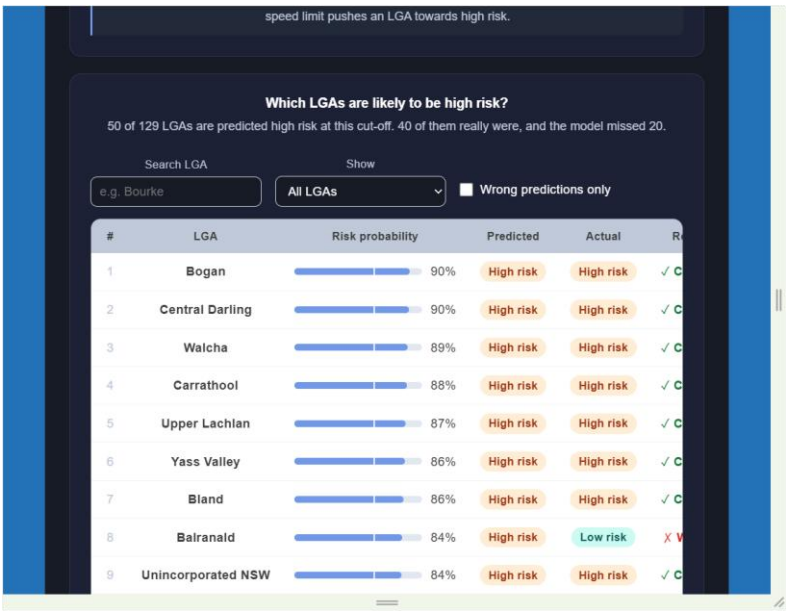


Figure 34 : Risk Predictor Responsiveness 3

Mobile responsiveness (550px)

For screens with a width of 550px and below, further adjustments are made to reduce the padding around the page sections and chart containers. The ROC Curve chart and Confusion Matrix shrink further to fit the limited screen width. The LGA table is also compressed to fit the smaller display. Although some columns become narrower, the table remains usable, with users able to scroll horizontally to view the remaining columns.

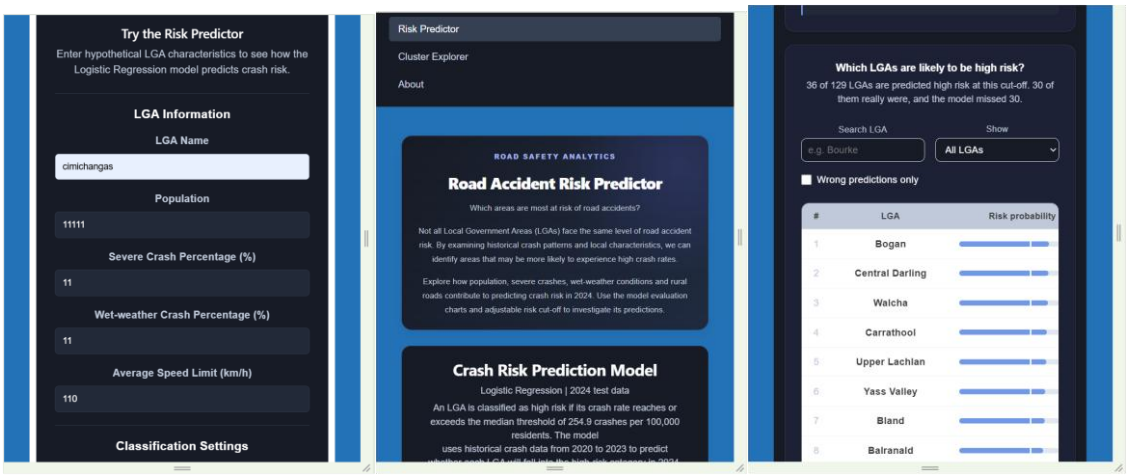


Figure 35: Risk Predictor Responsiveness 4

6. Cluster Explorer Page

This section presents the Cluster Explorer page of the Road Safety website. It covers the page layout and filters, backend API integration, data processing and CSV files, K-Means and DBSCAN clustering models, PCA, interactive D3.js visualisations, and responsive chart design. These features allow users to explore road-safety patterns across New South Wales (NSW) Local Government Areas (LGAs).

6.1 Page Layout and Filters

6.1.1 Page Layout

The cluster explorer page is designed to give users an interactive interface to explore road safety patterns across New South Wales (NSW) Local Government Areas (LGAs). The web page begins with an introduction to the purpose of this page, followed by the interactive visualisation and clustering results.

In the introductory section, it would state a small description of exploring road safety patterns. Then it would show three cards, each describing the available cluster. Each card presents the main road safety characteristics associated with its cluster, which helps users to understand the differences between the identified groups. Users would also see a small description on how to explore the data by using filters. Users are encouraged to start with the scatter plot to observe the distributions of the LGA-year before using the Cluster Profile Bar chart to compare the different road safety characteristics of each cluster.

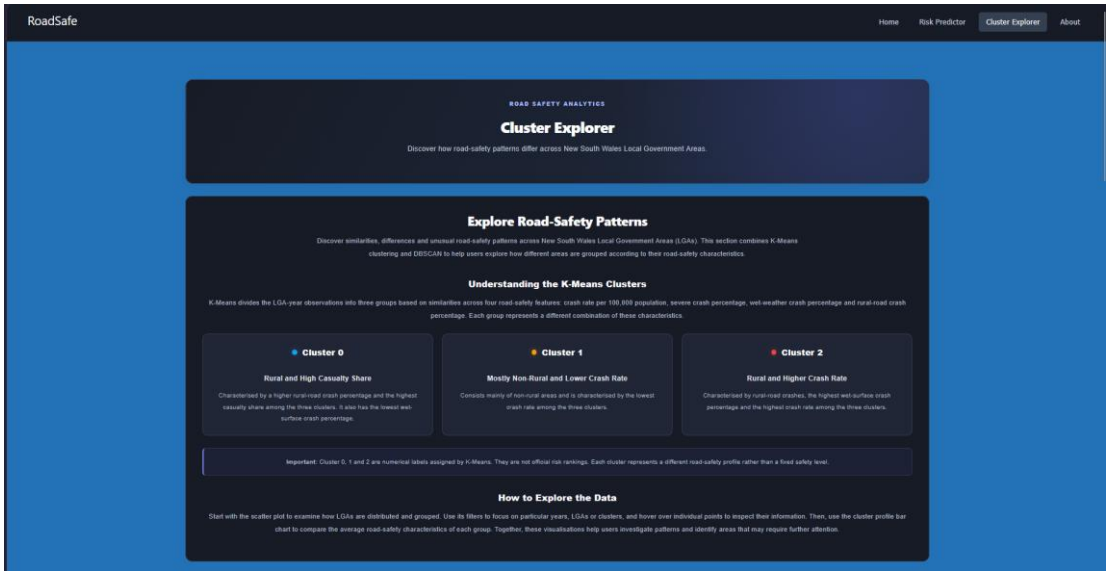


Figure 36 : Cluster explorer page

6.1.2 Scatter Plot Filters

The scatter plot chart has been implemented with several filtering controls that allow users to narrow down the displayed LGA-year observations. These filters allow multiple selections and value input.

The filters include:

- Year – Allows users to select one or multiple years to focus on
- LGA – Allows users to select one or multiple LGAs to focus on
- K-Means Cluster – Allows users to select one or more clusters to examine records belonging to a particular group
- DBSCAN- Provides two selections: noise and normal, allowing users to display records that are noise or not.
- PCA Coordinate Ranges - Provides four number input fields for setting the minimum and maximum values of PC1 and PC2

A reset button for the filters was also added to clear all selected option restoring the filters to it default view.

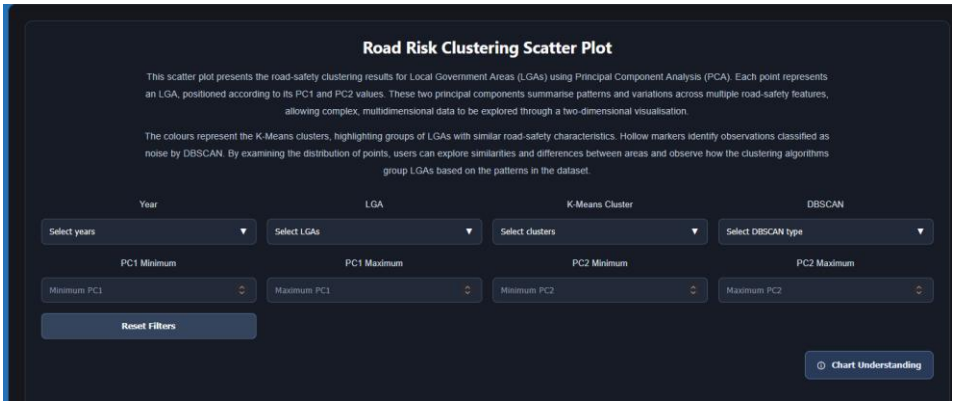


Figure 37: Scatter plot filter

6.1.3 Cluster Profile Bar Chart Selection

The cluster Profile Bar Chart includes buttons that allow users to switch between the different cluster views to look at the average of four road characteristics features across all available clusters.

When all clusters are selected, the chart displays the average values of our road-safety features across the three cluster profiles: crash rate per 100,000 population, severe crash percentage, wet-weather crash percentage, and rural-road crash percentage. So when an individual cluster is pressed, it shows the average values of all four features for only the selected clusters. An insight panel was added to provide a brief explanation of what the bar chart represents.

This chart allows users to compare the average road safety characteristics of the three K-means clusters, examine the differences between their crash –related features, and identify the patterns that distinguish each group.

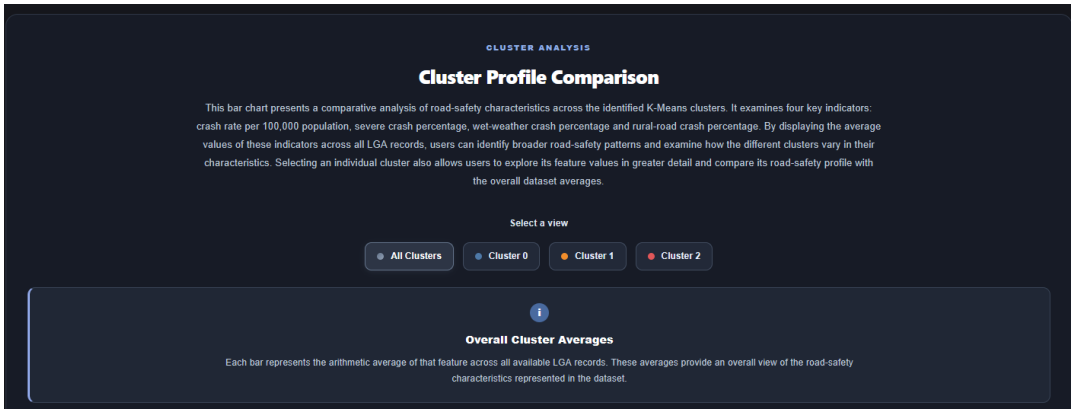


Figure 38: Bar Chart filter

6.2 Backend API Intergration

The Cluster Explorer webpage retrieves its clustering data from the backend API endpoints implemented in cluster.py using FASTAPI. These endpoints provide the dataset required for the scatter plot and the bar chart.

The two API endpoints are:

- GET /api/clusters/pca – Retrieves the data from lga_cluster_points.csv, which contains the LGA-year records, PCA coordinates, and clustering information required by the Scatter Plot.
- GET /api/clusters/profiles – Retrieves the data from cluster_profiles.csv, which contains the average values of the four road-safety features for each K-Means cluster.

On the frontend, clusterexplorer.jsx was a Fetch API function that is used to send requests to both endpoints. The API base URL is constcructed using The API base URL is constructed using window.location.hostname with port 8000. This allows the frontend to connect to the backend using the current hostname.

So when the cluster explorer page is loaded, the useEffect() hook uses Promise.all() to fetch data from both API endpoints concurrently. The responses are then converted into JSON files and stored in React state called points and profiles. After that is the passed down to each chart. Where points is for the scatter plot (ScatterPlot.jsx), and profiles is for the Bar chart (ClusterProfile.jsx)

```
//chostname so frontend can connect to backend
const API_URL = `http://${window.location.hostname}:8000`;

function ClusterExplorer() {
  const [points, setPoints] = useState([]); //PCA CORRDINATES STORAGE
  const [profiles, setProfiles] = useState([]);// STORES AVERAGE VALUES USED
  const [loading, setLoading] = useState(true);//SEE IF API DATA IS LOADING
  const [error, setError] = useState(""); //ERROR MESSAGE CALLING WHEN API FA

  // This function retrive the sactter plot and cluster profile data from the
  useEffect(() => {
    async function loadData() {

      try {
        //fetch data from here
        const [pointsResponse, profilesResponse] = await Promise.all([
          fetch(`${API_URL}/api/clusters/pca`),
          fetch(`${API_URL}/api/clusters/profiles`),
        ]);
```

Figure 39: API intergration

6.3 Data Processing and CSV Files

Two CSV files, lga_cluster_points.csv and cluster_profiles.csv, were created to provide data for both visualizations in the Cluster Explorer. Each CSV file has a different purpose; the first one provides LGA-year records for the Scatter Plot. The second one provides average values for the Cluster Profile Bar Chart.

6.3.1 Implementation of lga_cluster_points.csv

The lga_cluster_points.csv file was created by modifying the original dataset from assignment 2, which is lga_yearly_aggregates_clustering.csv. Then this file was copied using data. copy so additional columns could be added by changing the DataFrame.

A total of three columns were added. PC1 and PC2 columns were added to store the first two PCA coordinates, which determine the LGA-year record on the two-dimensional scatter plot. Next is the noise column, which identifies records classified as noise by DBSCAN. This is done by checking where the DBSCAN column is equal to –1.



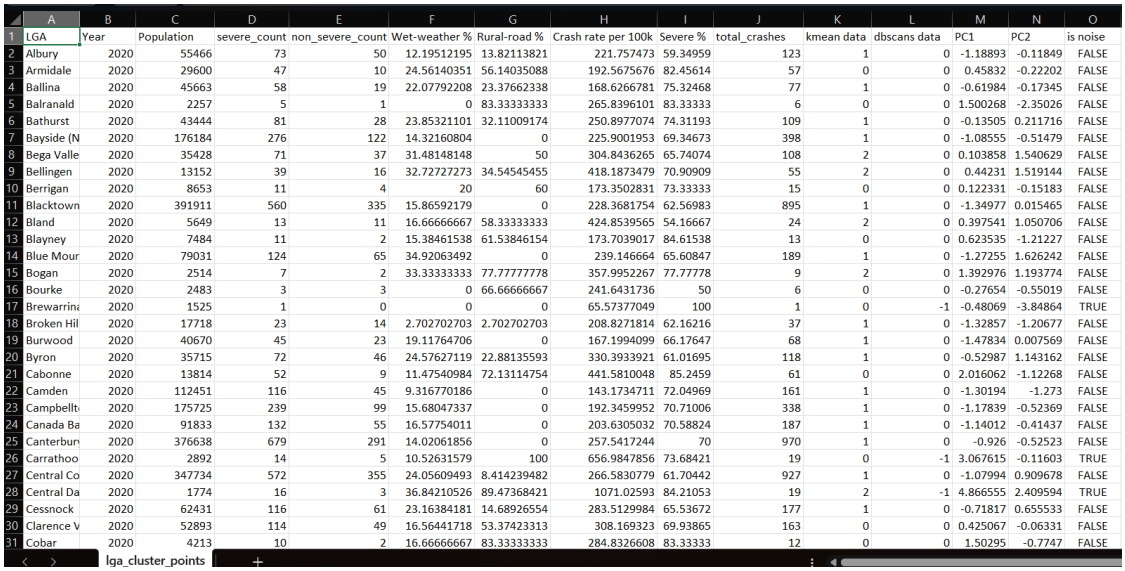
Figure 40: Modified CSV

After this, the completed DataFrame would then be exported as lga_cluster_points.csv. So currently, it has all the features needed to provide the Scatter Plot with PCA coordinates and clustering information needed to display, which helps determine the difference between k-means clusters and DBSCAN noise points. This CSV contains the following features:

- LGA
- Year
- Population
- Severe count
- Non-severe count
- Wet weather %
- Rural road %
- Crash Rate per 100k
- Severe%
- Total_crashes
- K-means data
- Dbscan
- PC1
- PC2
- Isnoise

These values are used to create the Road Risk Clustering Scatter Plot, and they allow us to compare clusters with different features, such as:

- LGA
- Year
- K-means Cluster
- DBSCAN
- PC1 maximum and minimum values
- PC2 maximum and minimum values



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	LGA	Year	Population	severe_count	non_severe_count	Wet-weather %	Rural-road %	Crash rate per 100k	Severe %	total_crashes	kmean data	dbscans data	PC1	PC2	is noise
2	Albury	2020	55466	73	50	12.19512195	13.82113821	221.757473	59.34959	123	1	0	-1.18893	-0.11849	FALSE
3	Armidale	2020	29600	47	10	24.56140351	56.14035088	192.5675676	82.45614	57	0	0	0.45832	-0.22202	FALSE
4	Ballina	2020	45663	58	19	22.07792208	23.37662338	168.6266781	75.32468	77	1	0	-0.61984	-0.17345	FALSE
5	Balranald	2020	2257	5	1	0	83.33333333	265.8396101	83.33333	6	0	0	1.500268	-2.35026	FALSE
6	Bathurst	2020	43444	81	28	23.85321101	32.11009174	250.8977074	74.31193	109	1	0	-0.13505	0.211716	FALSE
7	Bayside (N	2020	176184	276	122	14.32160804	0	225.9001953	69.34673	398	1	0	-1.08555	-0.51479	FALSE
8	Bega Valle	2020	35428	71	37	31.48148148	50	304.8436265	65.74074	108	2	0	0.103858	1.540629	FALSE
9	Bellingen	2020	13152	39	16	32.72727273	34.54545455	418.1873479	70.90909	55	2	0	0.44231	1.519144	FALSE
10	Berrigan	2020	8653	11	4	20	60	173.3502831	73.33333	15	0	0	0.122331	-0.15183	FALSE
11	Blacktown	2020	391911	560	335	15.86592179	0	228.3681754	62.56983	895	1	0	-1.34977	0.015465	FALSE
12	Bland	2020	5649	13	11	16.66666667	58.33333333	424.8539565	54.16667	24	2	0	0.397541	1.050706	FALSE
13	Blayney	2020	7484	11	2	15.38461538	61.53846154	173.7039017	84.61538	13	0	0	0.623535	-1.21227	FALSE
14	Blue Mour	2020	79031	124	65	34.92063492	0	239.146664	65.60847	189	1	0	-1.27255	1.626242	FALSE
15	Bogan	2020	2514	7	2	33.33333333	77.77777778	357.9952267	77.77778	9	2	0	1.392976	1.193774	FALSE
16	Bourke	2020	2483	3	3	0	66.66666667	241.6431736	50	6	0	0	-0.27654	-0.55019	FALSE
17	Brewarrina	2020	1525	1	0	0	0	65.57377049	100	1	0	-1	-0.48069	-3.84864	TRUE
18	Broken Hil	2020	17718	23	14	2.702702703	2.702702703	208.8271814	62.16216	37	1	0	-1.32857	-1.20677	FALSE
19	Burwood	2020	40670	45	23	19.11764706	0	167.1994099	66.17647	68	1	0	-1.47834	0.007569	FALSE
20	Byron	2020	35715	72	46	24.57627119	22.88135593	330.3933921	61.01695	118	1	0	-0.52987	1.143162	FALSE
21	Cabonne	2020	13814	52	9	11.47540984	72.13114754	441.5810048	85.2459	61	0	0	2.016062	-1.12268	FALSE
22	Camden	2020	112451	116	45	9.316770186	0	143.1734711	72.04969	161	1	0	-1.30194	-1.273	FALSE
23	Campbellt	2020	175725	239	99	15.68047337	0	192.3459952	70.71006	338	1	0	-1.17839	-0.52369	FALSE
24	Canada Ba	2020	91833	132	55	16.57754011	0	203.6305032	70.58824	187	1	0	-1.14012	-0.41437	FALSE
25	Canterbur	2020	376638	679	291	14.02061856	0	257.5417244	70	970	1	0	-0.926	-0.52523	FALSE
26	Carrathoo	2020	2892	14	5	10.52631579	100	656.9847856	73.68421	19	0	-1	3.067615	-0.11603	TRUE
27	Central Co	2020	347734	572	355	24.05609493	8.414239482	266.5830779	61.70442	927	1	0	-1.07994	0.909678	FALSE
28	Central Da	2020	1774	16	3	36.84210526	89.47368421	1071.02593	84.21053	19	2	-1	4.866555	2.409594	TRUE
29	Cessnock	2020	62431	116	61	23.16384181	14.68926554	283.5129984	65.53672	177	1	0	-0.71817	0.655533	FALSE
30	Clarence V	2020	52893	114	49	16.56441718	53.37423313	308.169323	69.93865	163	0	0	0.425067	-0.06331	FALSE
31	Cobar	2020	4213	10	2	16.66666667	83.33333333	284.8326608	83.33333	12	0	0	1.50295	-0.7747	FALSE

lga_cluster_points

*Figure 41: **lga_cluster_points.csv***

6.3.2 Implementation of cluster_profiles.csv

This CSV was already implemented in Assignment 2, so there were no changes made to it. The CSV contained average values of four selected road safety features for each K-means cluster:

- Crash Rate per 100k
- Severe %
- Wet-weather %
- Rural-road %.

These values are used to create the Cluster Profile Comparison Bar Chart, and it allows it to compare the road safety characteristics of Cluster 0, Cluster 1 and Cluster 2.

	A	B	C	D	E	F
1	kmean data	Crash rate per 100k	Severe %	Wet-weather %	Rural-road %	
2	0	316	76.28	11.35	63.73	
3	1	217.96	67.15	18.42	12.63	
4	2	433.16	67.95	26.93	67.91	
5						
6		GROUP 10 COS30049				
7						

*Figure 42: **cluster_profiles.csv***

6.4 K-Means, DBSCAN and PCA Models

The Cluster Explorer uses K-Means clustering, DBSCAN and Principal Component Analysis (PCA) to analyse road safety patterns across all NSW LGAs.The two research questions addressed by K-Means and DBSCAN models were established in Assignment 2 and carried forward into this implementation.

6.4.1 K-means clustering

K-means is a machine learning technique that groups data records based on similarities in their selected crash features. In this project, K-means divides data into three clusters.

Each of these clusters represents records with similar road crash characteristics:

- Cluster 0 - **Rural and High Casualty Share**
- Cluster 1 - **Mostly Non-Rural and Lower Crash Rate**
- Cluster 2 - **Rural and Higher Crash Rate**

The k-means model is made to answer one question:

Which NSW Local Government Areas (LGAs) have similar road crash patterns based on the selected crash features?

6.4.2 DBSCAN clustering

DBSCAN is a density-based technique that identifies groups of closely related data records, and unlike k-means, it can identify records that do not belong to dense groups. These records are known as noise points and will indicate that there is an unusual crash pattern if detected.

The DBSCAN model is made to answer one question:

Which LGA-year records show unusual crash patterns that do not fit into any identified dense group?

6.4.3 Principal Component Analysis (PCA)

PCA in our assignment is used to reduce the dimensionality of the data so that the clustering results can be placed on a scatter plot, which is two-dimensional. Since the original dataset contains multiple crash features, displaying all features together would be difficult. So, PCA turns these features into principal components, PC1 AND PC2, which are used on the scatter plot as the horizontal and vertical axes, allowing users to explore the distribution of the record and observe the patterns between the clusters.

6.5 React D3.js Clustering Charts

The Cluster Explorer page contains two visluations which is the scatter plot and the Bar chart. Both of these charts are integrated into the React frontend and use D3.js to create and present the chart for road safety clustering results. Each chart is implemented separately in its own respective JSX and CSS files to maintain independence.

6.5.1 Scatter Plot (ScatterPlot.jsx)

The Scatter Plot is implemented in ScatterPlot.jsx.It uses D3.js to display individual LGA-year records in a two–dimensional coordinate system based on the PCA values, which are PC1 and PC2. These values are used to determine the horizontal and vertical positions for each dot on the plot, which represents an individual LGA-year record. This chart uses data from lga_cluster_points.csv, which contains 14 different features.

This visualisation also uses K-means labels to distinguish records belonging to which cluster. Moreover, there is also a label (donut shape) that shows the records that are classified as noise under DBSCAN. This allows users to examine the distribution of K-means clusters and noise records that DBSCAN did not assign to a dense cluster. Filters were added to allow them to narrow down the records, allowing the users to find the data they need. This makes it easier for them to explore specific LGA-year records and take note of the results they want.

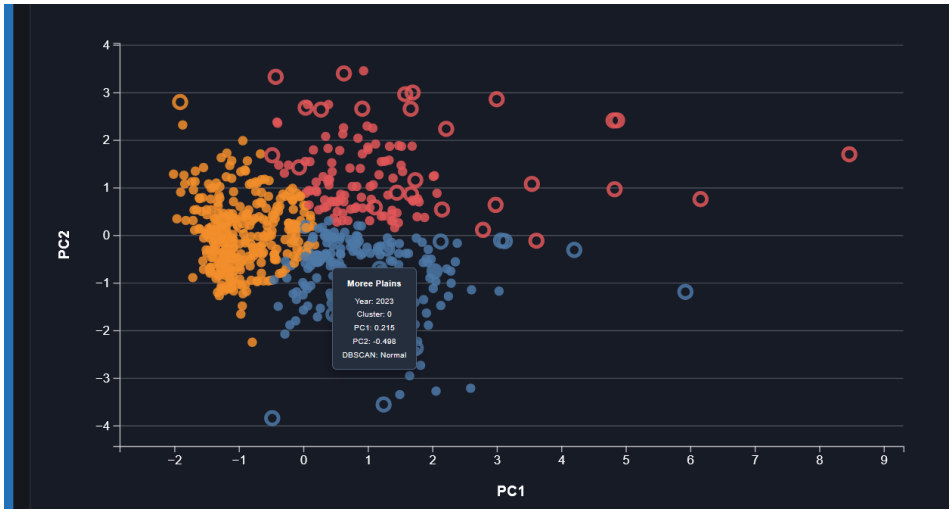


Figure 43: Scatter plot dots and tooltip

6.5.2 Bar Chart (ClusterProfile.jsx)

The clustering Bar Chart is implemented in ClusterProfile.jsx. It uses D3.js to display the average values for all four selected road features across the 3 clusters

This chart uses the data from cluster_profiles.csv to compare all four features. Each cluster is represented by the average value of each feature, which allows users to compare the four road safety characteristics of Clusters 0, 1, and 2. This visualisation provides a way to examine the differences of all three K-means clusters based on their average.

All for the all cluster seletction i had to create a fucntion that combines the data from all three K-Means cluster profiles. The function uses D3.js d3.mean() to calculate the average value of each of the four road-safety features across the three clusters. These calculated averages are then displayed in the bar chart, allowing users to compare the overall average values of all four features.

```
// Calculates the average value of each feature across all LGA records.
const averageProfile = useMemo(() => {
  return FEATURES.map((feature) => ({
    feature,
    value: d3.mean(
    chartData,
    (row) => row[feature]
    ) ?? null,
  }));
}, [chartData]);
```

Figure 44: Calculation for mean

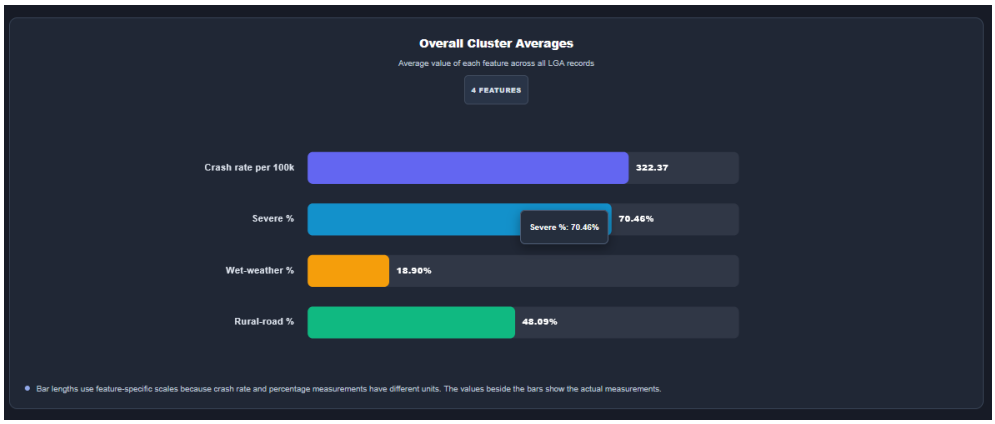


Figure 45: Bar Chart averages features

6.5.3 Chart styling

So both the charts use a similar dark theme design to maintain a uniform appearance across all the webpages. CSS was used to control the layout, colours, and the overall appearance of the chart components.

Scatter Plot (ScatterPlot.css)

This chart was designed to make sure the plotted data and chart elements were easier to distinguish. All the filter inputs were styled to match the overall theme and design of the charts. Additional hover transitions also provide visual feedback when users interact with the controls.

Designs were also added to the scatter plot, which included styled axes, grid lines, and a legend to help users interpret the PCA coordinates and distinguish the K-means cluster from DBSCAN noise. Moreover, when a user hover over a dot, a tooltip pops out whch provide addtional information about the individual record.

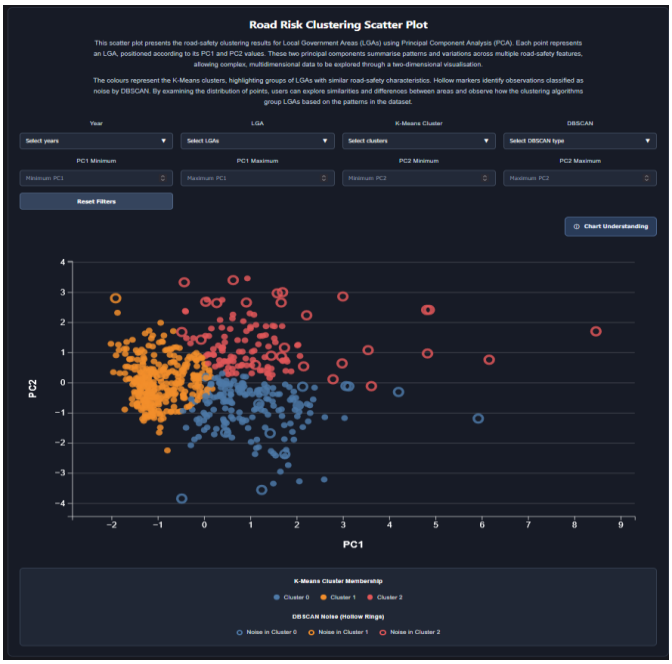


Figure 46: Full road risk scatter plot

Bar chart (ClusterProfile.jsx)

This chart was also designed in the same theme to maintain consistentsit of all the website pages. It uses a horizontal bar chart, with different colours representing the four road safety features: Crash Rate per 100k, Severe %, Wet-weather % and Rural-road %.

Buttons were implemeted to allow users to swtich between diffirent clusters. Each buttom has been labelled with the cluster dtot clours and when it is select it would be highlighted . The hovering over a bar there will be a tooltip tht displays additional information.

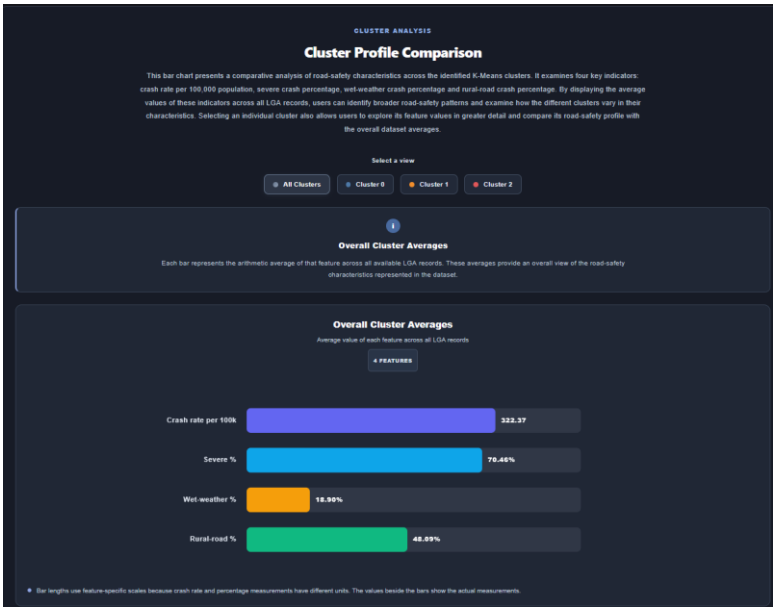


Figure 47: Full Cluster Profile Bar Chart

6.6 Responsive Chart Design

The cluster explorer uses CSS responsiveness to adjust the page layout and chart components according to different mobile devices. So, two breakpoints were implemented at 900px and 550px to provide a better layout for users on tablets and phones.

Tablet responsiveness(900px)

For Screens that are 900px and below, the page will be rearranged differently. For example, the three cluster information cards are rearranged into a single column. The page and chart section padding is reduced to make sure it is as big as possible for readability. For the scatter plot, the filter layout changes from four columns to two, allowing filter controls on a narrower screen. The bar chart also reduces its container padding and label font size to maintain readability.

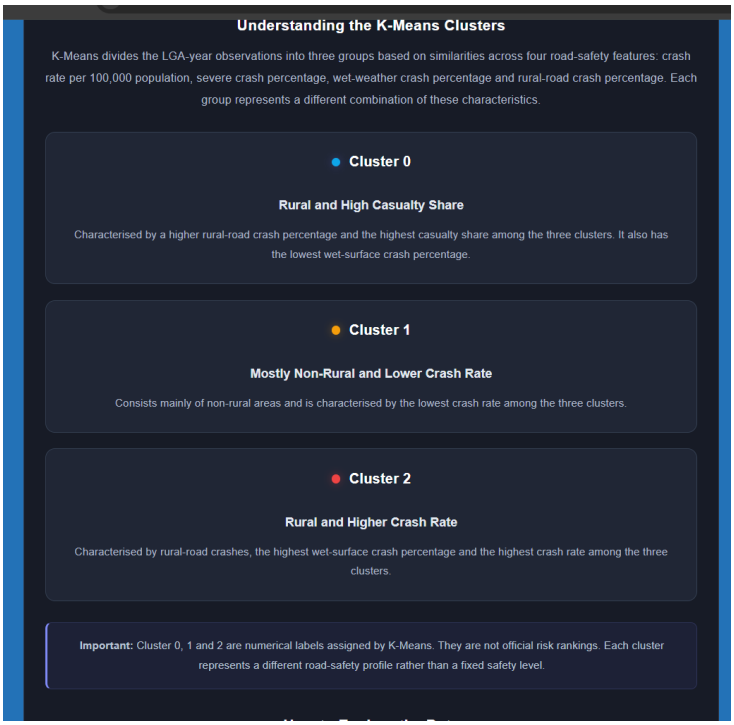


Figure 48: ClusterExplorer page responsiveness 1

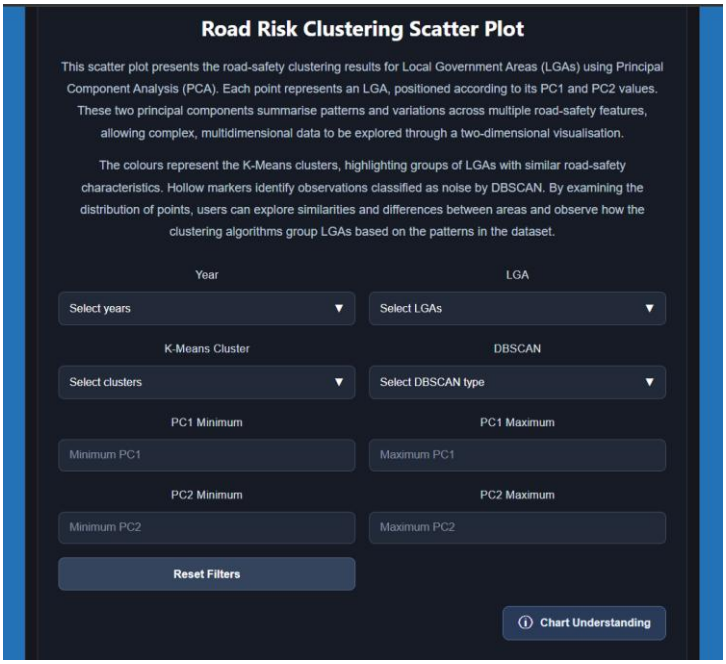


Figure 49: ClusterExplorer page responsiveness 2

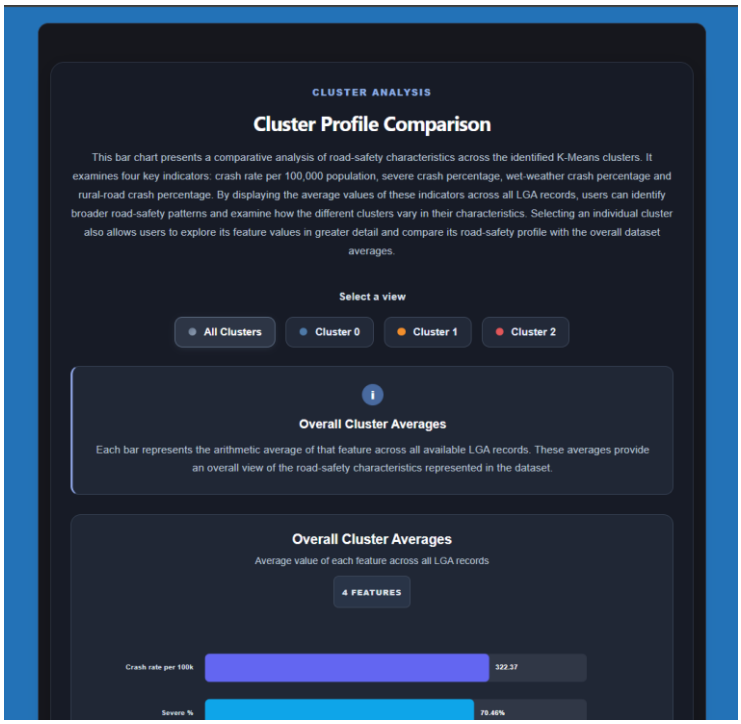


Figure 50: ClusterExplorer page responsiveness 3

Mobile Responsiveness (550px)

For screens with a width of 550px and below, further adjustments were made to reduce spacing and improve the use of limited screen space. A lot of parts that were reduced in size such as the page header, introduction section, chart containers and text sizes are reduced to fit smaller displays.

The scatter plot filters are rearranged into a single-column layout for better readability and touch responsiveness. The Chart Understanding button was also expanded because it was too small. For the Bar Chart, the cluster selection buttons were arranged into a two-column grid, and the chart heading and the record count are vertically stacked, while the chart panel padding and the note font size were reduced.

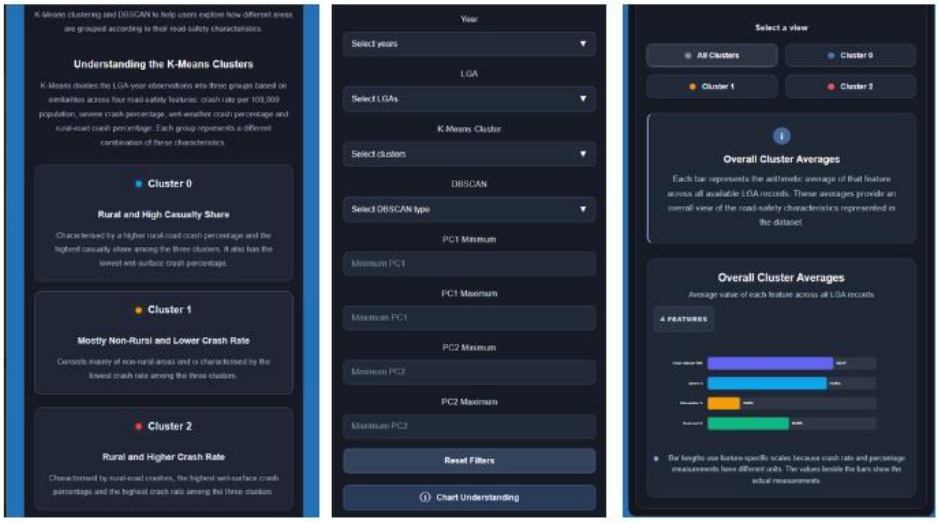


Figure 50: ClusterExplorer page responsiveness 4

Overall, these changes were necessary to allow the Cluster Explorers pages, sections, filters, and charts to be used on other devices while maintaining their usability and readability.

7. Back-End Design

The backend is built with FastAPI and handles requests from the React frontend, validates input, interacts with the AI models, and returns JSON data. To keep the code maintainable, the backend is split into three main folders: routers/ (handles HTTP routes), schemas/ (Pydantic models for validation), and services/ (model loading and prediction logic).

7.1 FastAPI Structure

- **Routers (**app/routers/**):** We created separate routers for different domains: lgas.py for data endpoints, risk.py for the Logistic Regression predictions and classification metrics, and cluster.py for the K-Means and PCA data.
- **Schemas (**app/schemas/**):** We used Pydantic to define the exact shape of incoming request bodies. For example, RiskRequest ensures that all four numerical features are provided and within valid ranges.
- **Services (**app/services/**):** This layer contains the actual business logic. risk_service.py and cluster_service.py are responsible for loading the .joblib files and running the prediction pipelines.

7.2 API Endpoints

Method	Endpoint	Purpose	Request Fields	Response Fields	Used By
GET	/lgas	Get list of all 129 LGAs	None	count, lgas (list)	Risk page
GET	/lgas/{lga_name}	Get historical data for one LGA	lga_name	lga, years_available, data	Risk page
POST	/api/classification/predict	Predict risk level for a hypothetical LGA	population, severe_pct, wet_weather_pct, avg_speed_limit	risk_level, probability, threshold_used	Risk page (form)
POST	/cluster	Predict cluster and PCA coordinates for an LGA	lga_name, year	cluster_id, cluster_name, pc1, pc2	Ended up not being used in the final version
GET	/api/clusters/pca	Get PCA coordinates for all 645 records	None	List of records with PC1, PC2, kmean data, is noise	Cluster page (scatter plot)
GET	/api/clusters/profiles	Get average features for each cluster	None	List of cluster profiles	Cluster page (bar chart)
GET	/api/classification/predictions	Get test predictions for the ROC chart	None	List of LGA, Year, y_true, y_prob	Risk page (charts)

GET	/api/classification/metrics	Get model evaluation metrics	None	accuracy, roc_auc, dummy_accuracy	Risk page (charts)
GET	/api/classification/importance	Get feature importance values	None	feature, importance_mean, importance_std	Risk page (charts)
GET	/api/classification/coefficients	Get model coefficients	None	feature, coefficient	Risk page (charts)

7.3 Server-side Validation

To prevent invalid data from reaching the AI models, we used Pydantic constraints in our schemas. For example, the RiskRequest schema uses Field(ge=0, le=100) for all percentage fields. If a user sends a request with severe_pct: 150, FastAPI automatically rejects it and returns a 422 Unprocessable Entity response with a clear error message explaining that the value must be less than or equal to 100.

7.4 Error Handling

We implemented error handling across the backend to ensure the frontend always receives a clean JSON response, even when something goes wrong:

- **404 Not Found**: If a user requests an LGA that does not exist in the CSV (e.g., GET /lgas/Atlantis), the API returns a 404 with {"detail": "LGA 'Atlantis' not found"}.
- **422 Unprocessable Entity**: Triggered automatically by Pydantic if the request body is missing fields or has invalid values.
- **500 Internal Server Error**: If a .joblib model file is missing or corrupted, the service layer catches the exception and returns a 500. To prevent the frontend from crashing entirely, the risk_service.py and cluster_service.py include a fallback that returns a "MOCK" prediction if the model cannot be loaded.

(Note: Refer to Appendix A to see the Swagger UI showing the full live API endpoints)

7.5 Startup and Performance

When the FastAPI server first starts up, it loads the .joblib model files and all the CSV data into memory. This happens in the services/ folder during the import stage, before the server even accepts requests.

The benefit of this is speed. Because the data is already sitting in RAM, the API doesn't have to read from the disk every time a user clicks a button or moves a slider. It just grabs the numbers it already has and sends a response back immediately. This is what allows the charts to update in real-time on the frontend without any noticeable delay.

8. AI Integration

8.1 Risk Model Integration

The Logistic Regression model was integrated into the Risk Predictor in two ways. The first uses the model's pre-generated results to display its performance, while the second allows users to test the trained model using their own inputs.

Model Results Integration (CrashRiskModel.jsx):

The Logistic Regression model was trained using road safety data from 2020 to 2023 and tested on 2024 data. Instead of running the model again whenever the page loads, the notebook exports its results into CSV files, which are retrieved through the FastAPI backend.

The `logreg_test_predictions.csv` file provides the predicted probabilities and actual risk labels used by the ROC Curve, Confusion Matrix and LGA Predictions Table. Other CSV files provide the model's evaluation metrics, feature importance values and coefficients.

The frontend uses these results to display the model's performance through interactive D3.js charts. Users can also adjust the probability threshold to see how the model's classifications, accuracy statistics and LGA predictions change without retraining the model.

Live Model Prediction (RiskPredictorForm.jsx):

A separate form allows users to test the actual trained Logistic Regression model using hypothetical LGA characteristics. Users enter population, severe crash percentage, wet-weather crash percentage and average speed limit.

When submitted, the form sends these values to the FastAPI backend through the `/api/classification/predict` endpoint. The backend loads the trained Scikit-Learn pipeline, which applies StandardScaler to the inputs before passing them into the Logistic Regression model.

The model uses `predict_proba()` to calculate the probability of the LGA being high risk. The backend then compares this probability against the default threshold of 0.5 and returns the predicted risk level and probability to the frontend for display.

Integration: React handles the interactive interface and form inputs, FastAPI connects the frontend to the model, and Scikit-Learn performs the actual predictions. This allows users to explore the model's existing results and test new inputs through the same Risk Predictor page.

8.3 Integration Pipeline

The integration pipeline is how the backend connects the frontend to the machine learning models. This logic lives in the services/ folder and handles everything from receiving the JSON request to sending back the final prediction.

Preprocessing: When a request hits the /api/classification/predict endpoint, FastAPI first validates the JSON body against the RiskRequest Pydantic schema. This ensures all four features (population, severe %, wet-weather %, and average speed limit) are present and within valid ranges. If a percentage is over 100, the request is rejected with a 422 error before it reaches the model. Once validated, the raw values are passed into the Scikit-Learn pipeline, which applies the StandardScaler to normalise the inputs.

The Model Call: The scaled features are passed into the Logistic Regression model using pipeline.predict_proba(). This returns a probability between 0 and 1, representing the likelihood that the LGA falls into the high-risk class.

Postprocessing: The backend compares the returned probability against the saved decision threshold (0.5). It formats the result into a clean JSON response, labelling it as "High" or "Low" risk, rounding the probability to 4 decimal places, and returning the original inputs alongside the prediction so the frontend can display them.

Out-of-Range Inputs: If the frontend sends invalid data (such as a negative population), the Pydantic schema intercepts the request and returns a 422 Unprocessable Entity error. This prevents the model from crashing or producing nonsensical predictions, and ensures the frontend always receives a helpful error message to display to the user.

9. Testing

9.1 Back-End Tests

We wrote an automated test suite using pytest and FastAPI's TestClient to verify the stability of the backend API. The tests cover validation, error handling, and data retrieval, ensuring the API responds correctly to both valid and invalid inputs.

Test Case	Input	Expected Result	Status
test_predict_valid	Valid JSON with 4 features (avg_speed_limit included)	200 OK, risk_level in response	Pass
test_predict_missing_field	JSON missing population	422 Unprocessable Entity	Pass
test_predict_invalid_percentage	severe_pct: 150	422 Unprocessable Entity	Pass
test_get_lga_valid	GET /lgas/Camden	200 OK, returns 5 years of data	Pass
test_get_lga_invalid	GET /lgas/Atlantis	404 Not Found	Pass
test_get_all_lgas	GET /lgas	200 OK, returns 129 LGAs	Pass
test_cluster_pca_data	GET /api/clusters/pca	200 OK, returns 645 records	Pass
test_classification_predictions	GET /api/classification/predictions	200 OK, returns predictions	Pass

(Note: The pytest terminal output is included in Appendix B)

9.2 Front-End and Responsive Checks

Navigation and Routing Testing

The frontend navigation was tested to ensure that users could access all four main sections of the application. The navigation links were checked to verify that they directed users to the correct pages, including Home, Risk Predictor, Cluster Explorer and About. The page navigation was also checked to ensure that the application displayed the correct content when switching between sections.

Home Page Testing

The Home page was tested to make sure that its content and layout were displayed correctly. The page heading, descriptions and other interface elements were checked for

proper alignment, spacing and readability. The page was also analyzed to identify any missing content or overlapping elements.

About Page Testing

The About page was tested to make sure that the project information was displayed correctly and organised into clear sections. The project overview, objectives, dataset information, machine learning methodologies and model evaluation details were checked. The page layout, headings, text alignment and spacing were also been analayzed to ensure that the information was readable and easy for users to understand.

Risk Predictor Page Testing

The Risk Predictor page was tested to ensure that the charts and various visualisations from CrashRiskModel.jsx as well as the form for RiskPredictorForm.jsx functioned correctly. Rigorous testing on the update function used in CrashRiskModel.jsx and RiskPredictorForm.jsx were tested to verify that the data updated accordingly as well.

Cluster Explorer Page Testing

The Cluster Explorer page was tested to ensure that the Scatter Plot, Cluster Profile Bar Chart and their interactive filters functioned correctly. Different filter selections and cluster options were tested to verify that the displayed data updated accordingly.

Responsive Layout Testing

The website was tested using different screen sizes to evaluate its responsiveness. The navigation menu, page content and interface elements were checked to ensure that they adjusted properly on smaller screens and on any devices. Particularly, was given to text readability, element alignment and horizontal scrolling to ensure that the website works across different devices.

The Cluster Explorer page was tested at different screen sizes to ensure that the charts, filters and buttons adjusted correctly. The layout was also checked for overlapping elements, cut-off content and readability issues.

10. Conclusion

10.1 Decisions

During the development of the Road Safety application, key technical and structural decisions were made to ensure the system remained functional, organized, and user-friendly. For the architecture, React was chosen to build a reusable, interactive frontend, while FastAPI was selected to manage backend communication between the user interface and the machine learning models. On the analytical side, Logistic Regression was used to classify binary road accident risk classification, alongside K-Means and DBSCAN to cluster Local Government Areas (LGAs) based on road safety characteristics, with Principal Component Analysis (PCA) reducing dimensionality for two-dimensional scatter plot visualization. Finally, the platform was structured into four distinct sections Home, Risk Predictor, Cluster Explorer, and About to separate core functionalities and streamline user navigation.

10.2 What the Site Achieves

The developed website successfully provides an interactive platform for exploring road safety information and understanding accident patterns across different Local Government Areas (LGAs). By combining machine learning techniques with data visualisation, it closes the gap between complex analytics and accessible public information, transforming raw data into practical insights. Instead of showing static statistics, the system empowers users to actively simulate accident risk scenarios and uncover hidden regional patterns across LGAs through real-time prediction and dimensional clustering. By integrating predictive modeling, pattern recognition, and transparency around model performance into a single functional platform, the application achieves its core objective of making data driven road safety analysis engaging for non technical users.

10.3 Future Enhancements

In the future, the Road Safety application could be improved by integrating real-time traffic information from sources such as roadside sensors, vehicle data, navigation applications and weather feeds. According to Intertraffic (2026), AI-powered traffic intelligence can combine information from different sources to identify unusual traffic patterns and predict potential safety risks before accidents occur. A similar approach could be introduced into our application to provide more updated road safety insights and help identify potential accident risks earlier. Furthermore, the system could explore more advanced predictive techniques and additional data sources to improve its analysis. However, these improvements would require reliable data, proper validation and consideration of possible prediction errors (Intertraffic, 2026).

Bibliography

Australian Bureau of Statistics (2026) *Regional population, 2024-25 financial year*. Available at: https://www.abs.gov.au/statistics/people/population/regional-population/latest-release (Accessed: 11 November 2026).

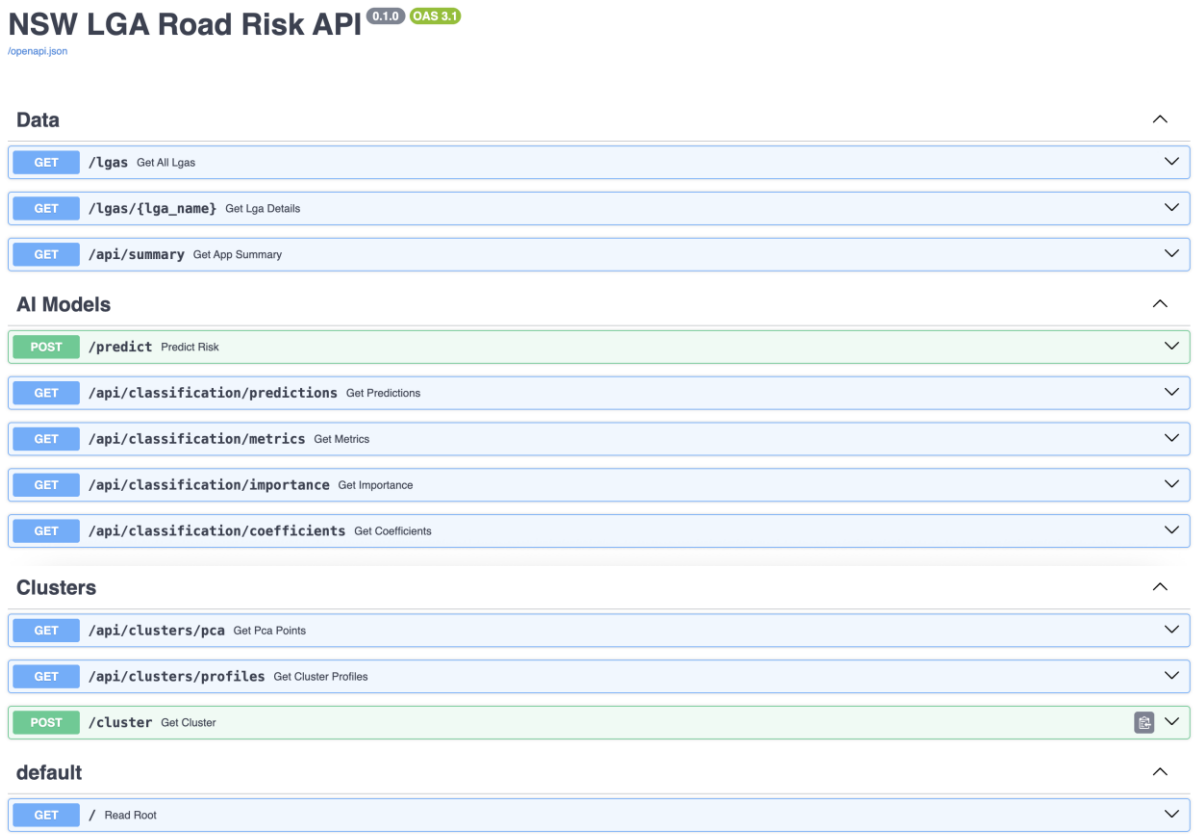
Transport for NSW (n.d.) *NSW Crash Data* [Dataset]. Available at: https://data.nsw.gov.au/data/dataset/2-nsw-crash-data (Accessed: 18 October 2026).

Intertraffic.com. (1970). *Predict and prevent : How AI traffic intelligence is reshaping road safety*. [online] Available at: https://www.intertraffic.com/news/road-safety/predict-and-prevent (Accessed 13 Nov. 2026).

Generative AI Acknowledgments

Members	Tool	What it is used for	How I checked and understood it
Lalmuanpuia	ChatGPT, Grammarly	Used ChatGPT to ask how to structure the full-stack project folders and to help debug a CORS configuration error. Grammarly was used for grammar and spelling checks.	I tested the CORS configuration and folder structure in Swagger to ensure it worked. I wrote and understand all the final backend code.
Livinggkumar A/L Supramaniam	ChatGPT, Claude AI	Used ChatGPT to help structure the React navigation layout and fix styling alignment issues on the Home and About pages. Claude AI was used to refine the wording and clarity of my report sections.	I tested the page navigation, layout responsiveness, and content display directly in the browser to confirm everything worked properly. I wrote and understand all the final frontend code for my pages and verified the report.
Jean-Luc William Clovis	Chatgpt, Grammerly	ChatGPT was used to help troubleshoot API and frontend connection issues, fix coding errors, and provide guidance on minor coding and styling improvements for the Cluster Explorer page. Grammarly was used to check grammar, spelling and sentence structure in my report.	I tested the API connections and Cluster Explorer features directly in the browser to ensure the data was retrieved and displayed correctly. I also checked the charts, filters and styling after making changes. I reviewed and tested the final code to understand how the features worked.
Teh Zhi Xiang	ChatGPT, Claude AI	Claude AI was used to help troubleshoot API connection issues, fixx coding errors as well as help provide some of the more complicated functions for CrashRiskModel.jsx I used ChatGPT to assist in writing my report for better clarity and grammar.	After generating the code, I first reviewed the code manually to ensure that I understand how it works. Then I inputed the code into my files to test them out and see how they work and make modifications to them make them fit the rest of the code.

Appendix A: The Swagger UI showing Live API Endpoints



Appendix B: Back-End Test Output

Below is the terminal output showing the successful execution of all 8 automated pytest tests for the FastAPI backend.

